

5팀 세미콜론 : 이태석, 문찬희, 박재현, 유승현

발표자 : 오세인

덕꾸들을 위한
취향 기반 큐레이션 중고거래 플랫폼



서비스 링크 <https://dukku.shop/>

스웨거 링크 <https://api.dukku.shop/swagger-ui/index.html#/>

목업 [덕꾸(DUKKU) - 취미 중고거래]

목업/admin [Semicolon/admin - 취미 중고거래]

덕쿠 DUKKU

덕쿠들을 위한
취향 기반 큐레이션 중고거래 플랫폼



5팀 세미콜론 : 이태석, 문찬희, 박재현, 유승현

발표자 : 오세인

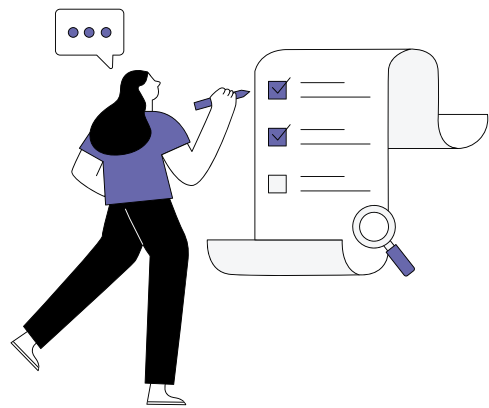
서비스 링크 <https://dukku.shop/>

스웨거 링크 <https://api.dukku.shop/swagger-ui/index.html#/>

목업 [덕쿠(DUKKU) - 취미 중고거래]

목업/admin [Semicolon/admin - 취미 중고거래]

Contents



01. 프로젝트 개요	02. 협업 및 개발 프로세스	03. MSA 아키텍처
04. DUKKU 시연 영상	05. 핵심 기능	06. 인프라 구축 및 시스템 고도화
07. 한계점 및 향후 개선 방향	08. 팀 구성과 역할	09. KPT 회고

프로젝트명: 덕쿠(dukku)

관심사 기반 중고거래 플랫폼 ‘덕쿠’

대규모 트래픽과 거래 정합성을 고려해 MSA 기반으로 설계

- 관심사 기반 중고거래 플랫폼
- 취미/덕질 카테고리 특화 C2C 서비스
- 안전한 거래 흐름을 고려한 구조 설계

선정 배경 (Why)

기존 중고거래의 한계

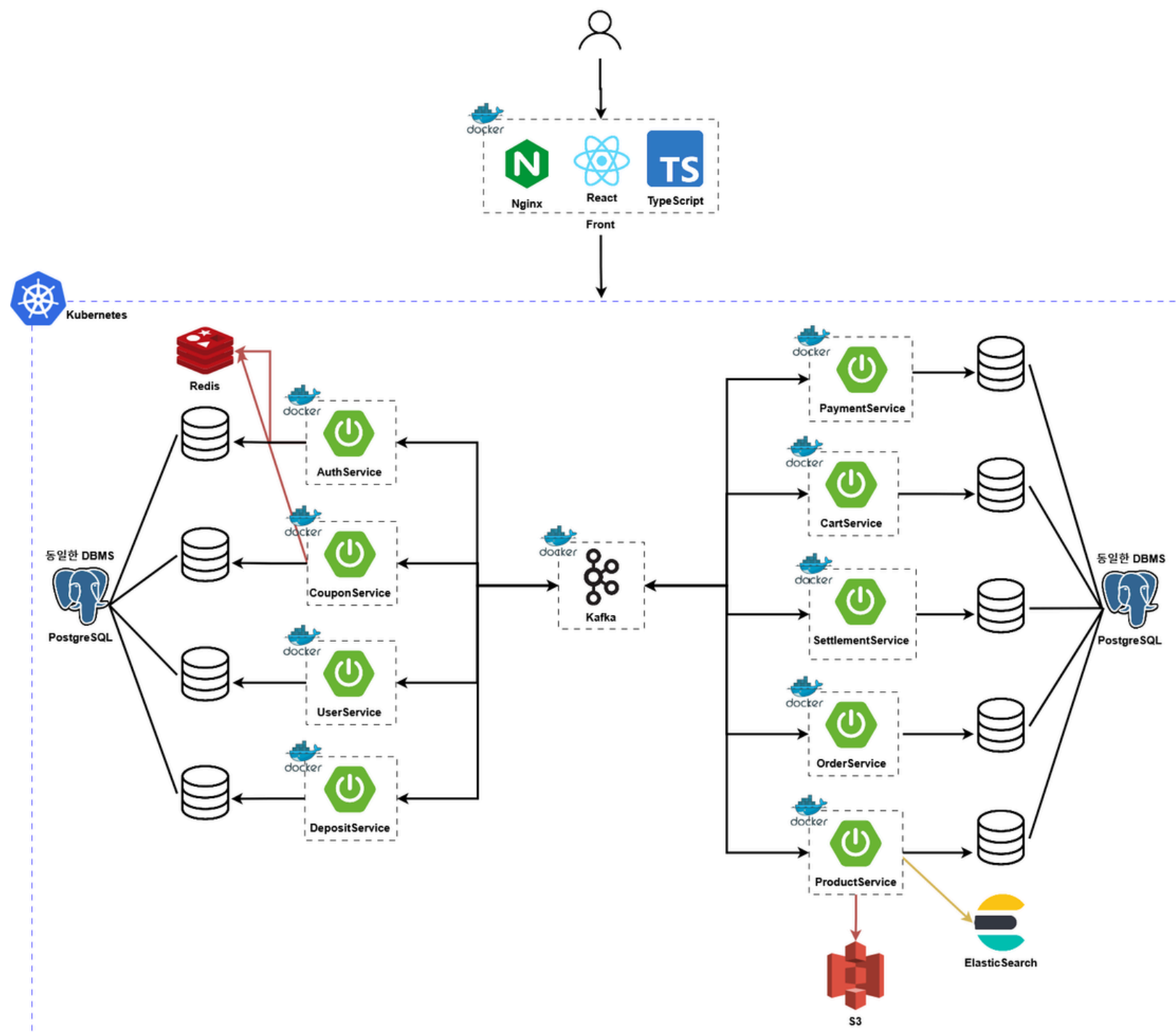
- 일반 카테고리 중심 → 취향 기반 커뮤니티 약함
- 거래-결제-정산 구조가 명확하지 않음
- 트래픽 증가 시 구조적 확장 어려움

DUKKU의 해결책

- 관심사 중심 커뮤니티 거래 구조
- 주문-결제-정산을 분리하여 안정적 처리
- 이벤트 기반 구조로 장애 전파 최소화



핵심 설계 요약



관심사 기반 상품 탐색

- 카테고리 세분화
- Elasticsearch 기반 검색 분리 → 확장성 확보

안전한 거래 정합성 설계

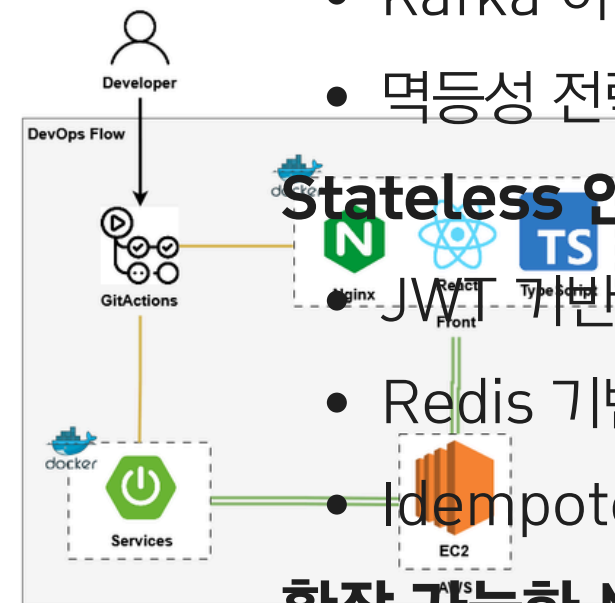
- 주문 → 결제 → 정산 도메인 분리
- Kafka 이벤트 기반 비동기 처리
- 멍등성 전략 적용으로 중복 처리 방지

Stateless 인증 및 보안 강화

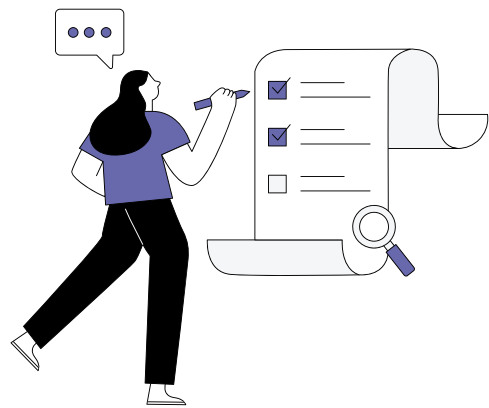
- JWT 기반 Access/Refresh 구조
- Redis 기반 Refresh Rotation + 절대 만료 정책
- Idempotency-Key + 분산락으로 중복 가입 방지

확장 가능한 MSA 구조

- 도메인별 서비스 분리
- 비동기 이벤트 처리로 결합도 최소화



Contents



01. 프로젝트 개요	02. 협업 및 개발 프로세스	03. MSA 아키텍처
04. DUKKU 시연 영상	05. 핵심 기능	06. 인프라 구축 및 시스템 고도화
07. 한계점 및 향후 개선 방향	08. 팀 구성과 역할	09. KPT 회고

개발 환경

1. Backend

- Spring Boot: 도메인 중심 API 서버
- JPA: 엔티티/리포지토리 기반 데이터 접근
- Redis: 토큰, 레이트리밋, 캐시
- Kafka: 도메인 이벤트 비동기 처리
- PostgreSQL: 트랜잭션 데이터 저장
- Elasticsearch(선택): 검색/자동완성/필터 성능 강화



2. Frontend

- React: SPA 기반 UI
- Antigravity: 컴포넌트 기반 디자인 시스템 적용



3. Infra

- Docker Compose: 로컬 개발/통합 테스트 환경 표준화
- k3s/Kubernetes: 서비스 배포 및 오토스케일 구조
- GitHub Actions: CI/CD 자동화
- 로그 수집: App Log → 수집기(예: Fluent Bit/Logstash) → 저장소(예: ELK/Cloud)



협업 프로세스

1. 브랜치 전략

- 도메인 단위 브랜치 네이밍 규칙

1 브랜치 네이밍 규칙

기본 규칙

- 소문자 사용
- 브랜치유형/도메인-설명 형태
- 필수: 기본 태그 + 도메인
- 도메인은 영어 사용

브랜치 유형

```

- feature
- refactor
- chore
- fix
- test
- build
- hotfix
- release

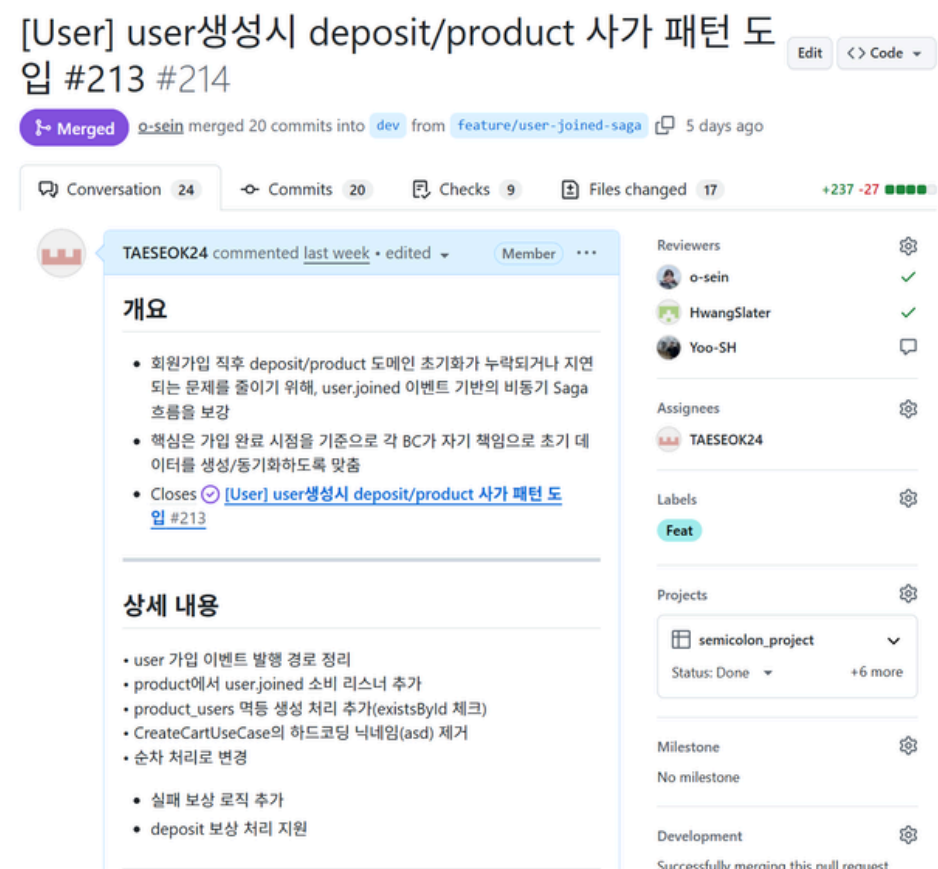
```

기능 개발 브랜치 예시

- feature/auth-login
- feature/product-search
- feature/order-cart
- feature/payment-deposit
- feature/settlement-batch

2. PR & 코드 리뷰 프로세스

- PR 템플릿 운영
- 최소 2인 승인 후 Merge
- 리뷰 시 비즈니스 로직 / 정합성 / 예외처리 중심 검토



3. 코드 컨벤션

- Git Commit Message
- Class Naming
- Code
- Pull-Request

Git Commit Message Convention

Class Naming Convention

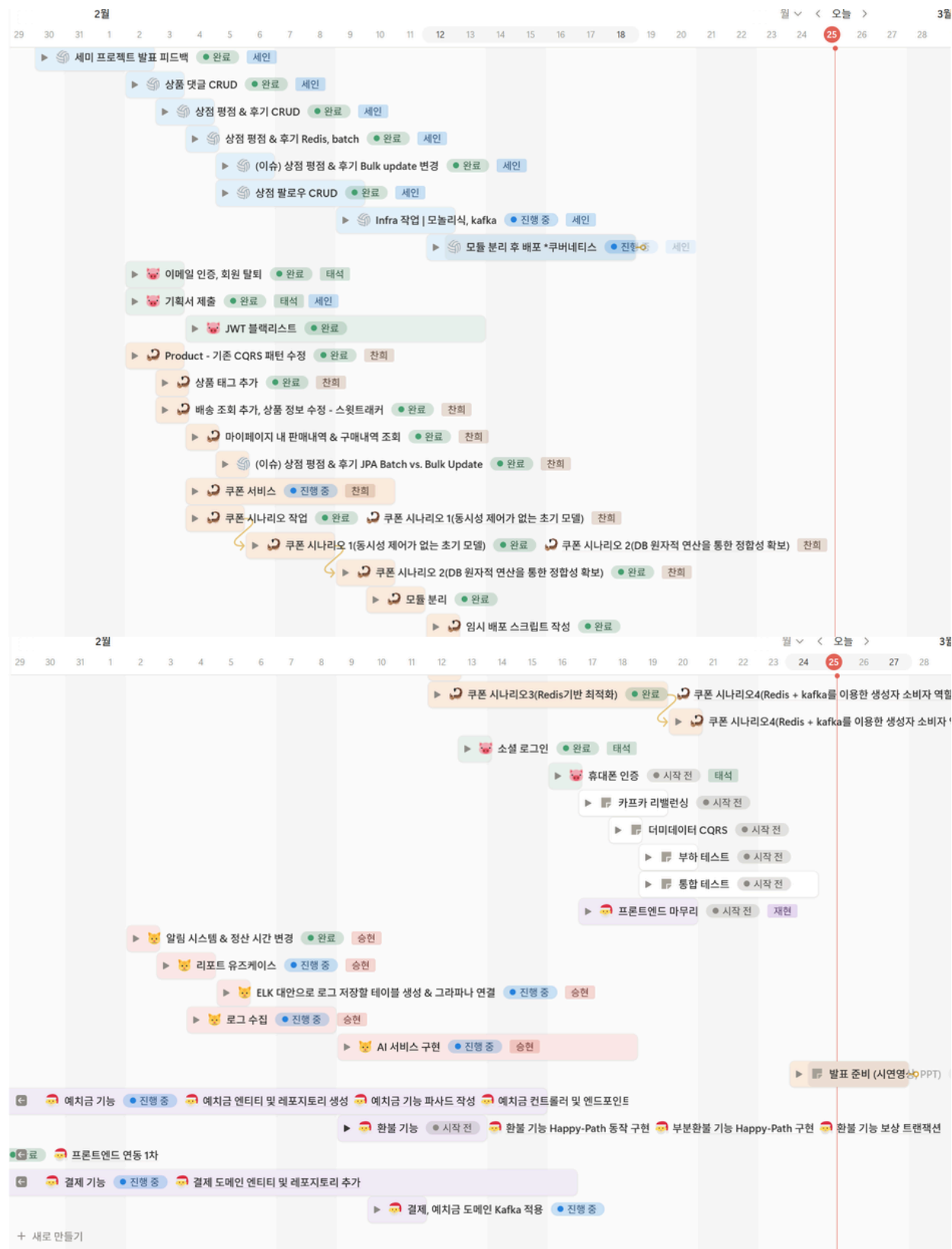
Code Convention

Pull-Request Convention



개발 일정

구분	1/29-2/2	2/3-2/7	2/8-2/12	2/13-2/18	2/19-2/23	2/24-2/27
구조 분석 / CQRS	진행	완료				
회원/상품 기능 보완	진행	진행	완료			
결제 고도화		진행	진행	완료		
환불/부분환불		설계	구현	완료		
이벤트 흐름 정리		진행	완료			
모듈/서비스 분리			진행	진행	완료	
Kafka 전환			진행	완료		
쿠폰/마이페이지			진행	완료		
AI 추천				진행	진행	완료
검색/벡터 구조				진행	완료	
로그/모니터링				진행	완료	
Kubernetes 배포					진행	완료
부하/동시성 테스트					진행	완료
통합 테스트					진행	완료
발표 준비						진행 - 완료



개발 일정

Phase 1. 도메인 설계 및 기반 구축 (1~2주차)

- 회고: 단순 CRUD 구현이 아닌, ERD 기반으로 도메인 경계를 엄격히 정의하는 데 집중
 - Insight: 초기 도메인 분리를 명확히 한 덕분에, 이후 Modular Monolith → MSA 확장 시 코드 간섭 없이 독립적 개발이 가능했음
- 설계 품질이 이후 확장성과 개발 속도에 직접적인 영향을 미침을 경험

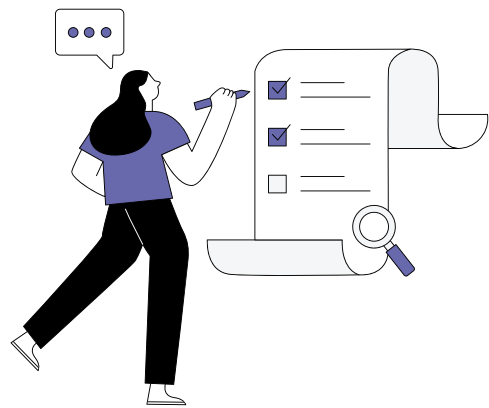
Phase 2. 핵심 로직 구현 및 기술적 전환 (3~4주차)

- 회고: 결제/환불 정합성과 Kafka 기반 이벤트 흐름 구현에서 예상보다 높은 복잡도 경험
 - Insight: 실패를 전제로 한 설계를 위해 Saga 패턴과 보상 트랜잭션을 도입
 - 일정 지연은 핵심 도메인 집중 개발 및 페어 프로그래밍으로 회복
- MSA 환경에서 정합성 확보는 기능 구현보다 설계 전략이 더 중요함을 체감

Phase 3. 안정성 확보 및 고도화 (5~6주차)

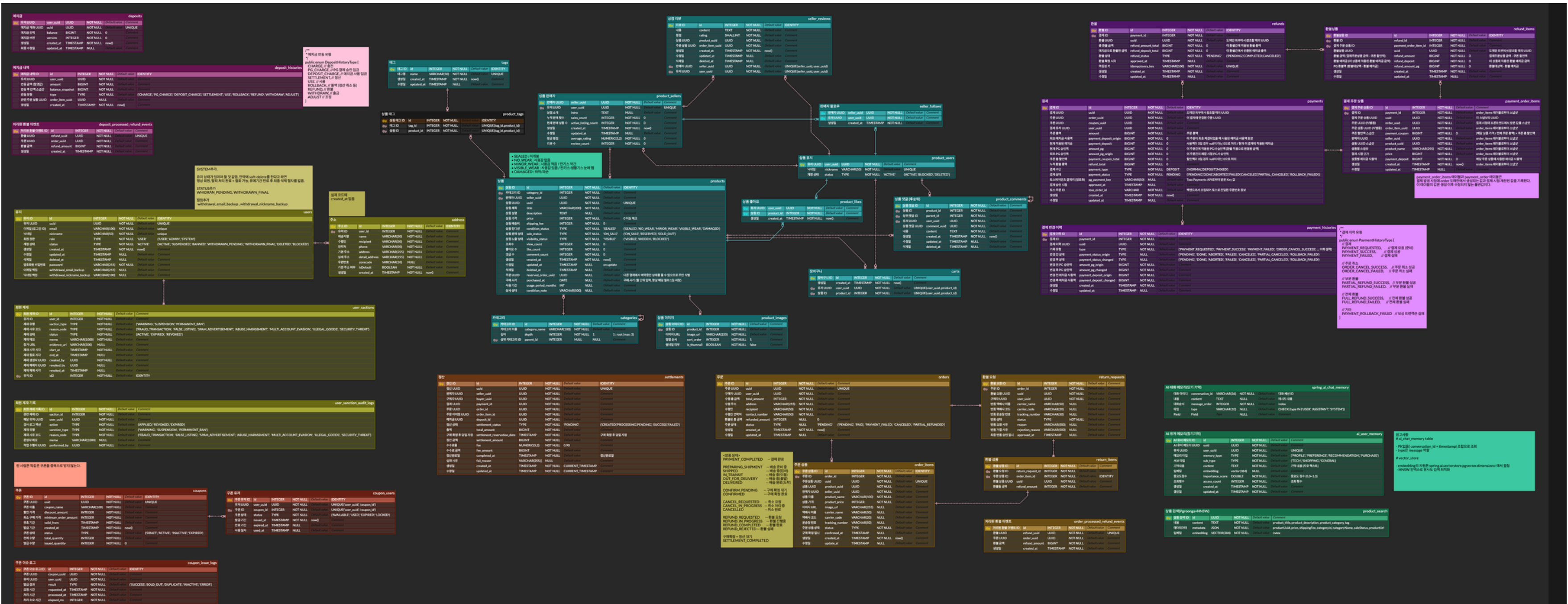
- 회고: 검색 성능 개선과 Kubernetes 기반 배포 환경 구축을 병행
 - Insight: CQRS를 적용해 읽기 트래픽을 분리하고, K8s 기반 무중단 배포 환경을 구축하여 서비스 가용성 확보
- 성능과 운영 안정성을 동시에 고려한 아키텍처로 고도화

Contents



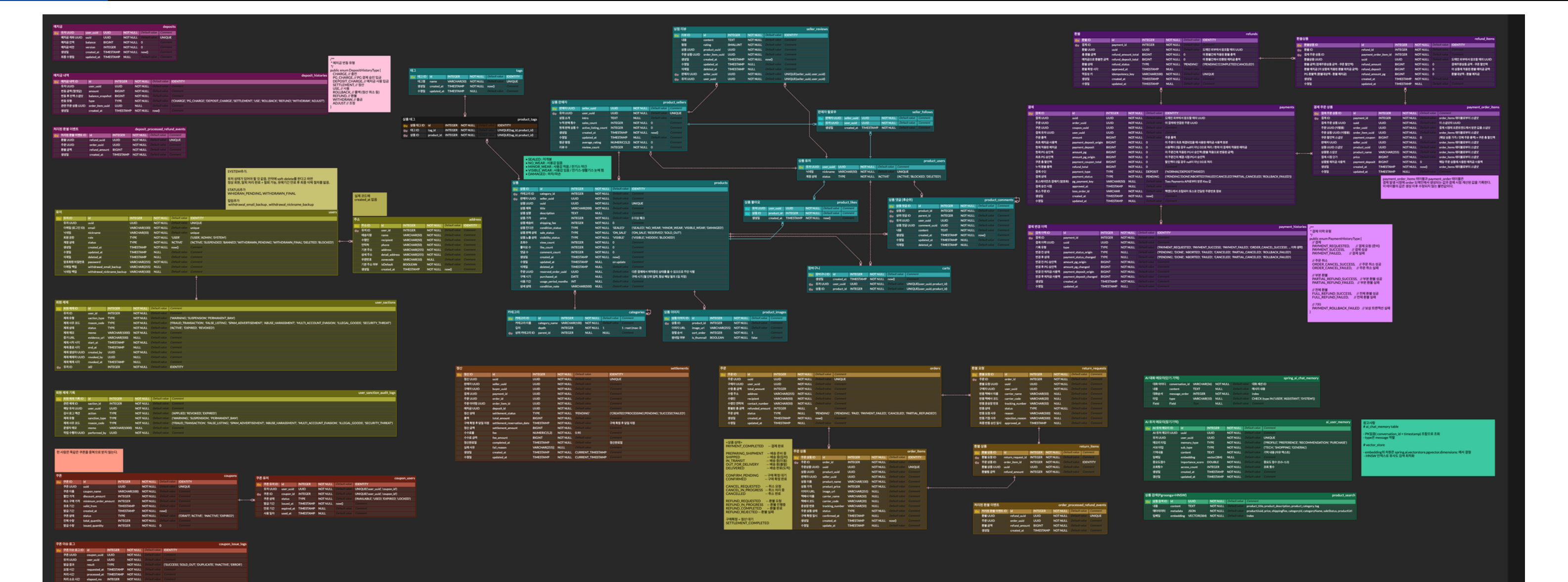
01. 프로젝트 개요	02. 협업 및 개발 프로세스	03. MSA 아키텍처
04. DUKKU 시연 영상	05. 핵심 기능	06. 인프라 구축 및 시스템 고도화
07. 한계점 및 향후 개선 방향	08. 팀 구성과 역할	09. KPT 회고

03 MSA 아키텍처



- 도메인 단위 책임 분리를 통한 정합성 확보
- 이벤트 기반 비동기 처리로 트랜잭션 결합도 최소화
- 서비스 간 직접 호출을 지양하고 Kafka 기반으로 확장성 확보

03 MSA 아키텍처



주요 담당 도메인 및 기술 스택

Auth/User: JWT 및 Redis Blacklist를 활용한 무상태(Stateless) 보안 체계 구축

Payment/Deposit: SAGA 패턴을 적용한 이벤트 기반 주문 - 결제 프로세스 구현 및 실패 시 안정적 복원을 위한 보상 트랜잭션 구조 구현

AI/Monitoring/Settlement: 사용자 취향 분석 데이터 처리 및 월 단위 정산 배치(Batch) 프로세스 안정성 확보

Order/Coupon: 주문 시점의 상품 데이터 스냅샷 보존 및 동시성 이슈를 고려한 실시간 쿠폰 발급 로직

Product/Infra: Elasticsearch 기반 고성능 검색 인덱싱 및 Kubernetes를 통한 무중단 배포 환경 관리

Auth/User: JWT 및 Redis Blacklist를 활용한 무상태(Stateless) 보안 체계 구축

Payment/Deposit: SAGA 패턴을 적용한 이벤트 기반 주문 - 결제 프로세스 구현 및 실패 시 안정적 복원을 위한 보상 트랜잭션 구조 구현

AI/Monitoring/Settlement: 사용자 취향 분석 데이터 처리 및 월 단위 정산 배치(Batch) 프로세스 안정성 확보

Order/Coupon: 주문 시점의 상품 데이터 스냅샷 보존 및 동시성 이슈를 고려한 실시간 쿠폰 발급 로직

Product/Infra: Elasticsearch 기반 고성능 검색 인덱싱 및 Kubernetes를 통한 무중단 배포 환경 관리

전체 시스템 구조

도메인 분리

- User/Order/Payment/Settlement/Deposit/Coupon/Product/Atuh/AI

핵심 거래 흐름

- Client → Order → Payment → Settlement
→ order.completed 이벤트 발행

실패 보상 흐름

- Payment 실패 → Deposit Rollback → ORDER_FAILED 확정

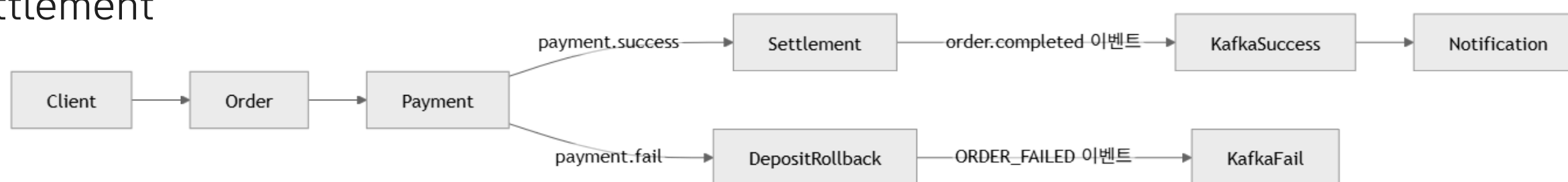
트랜잭션 경계

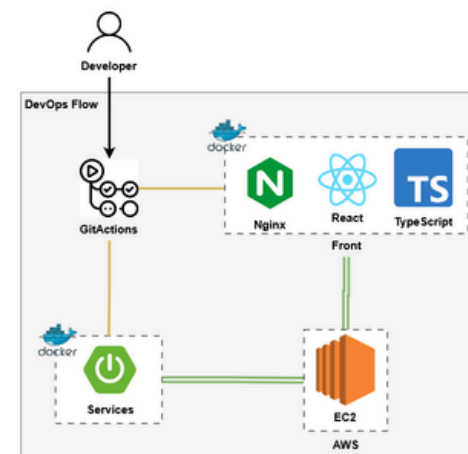
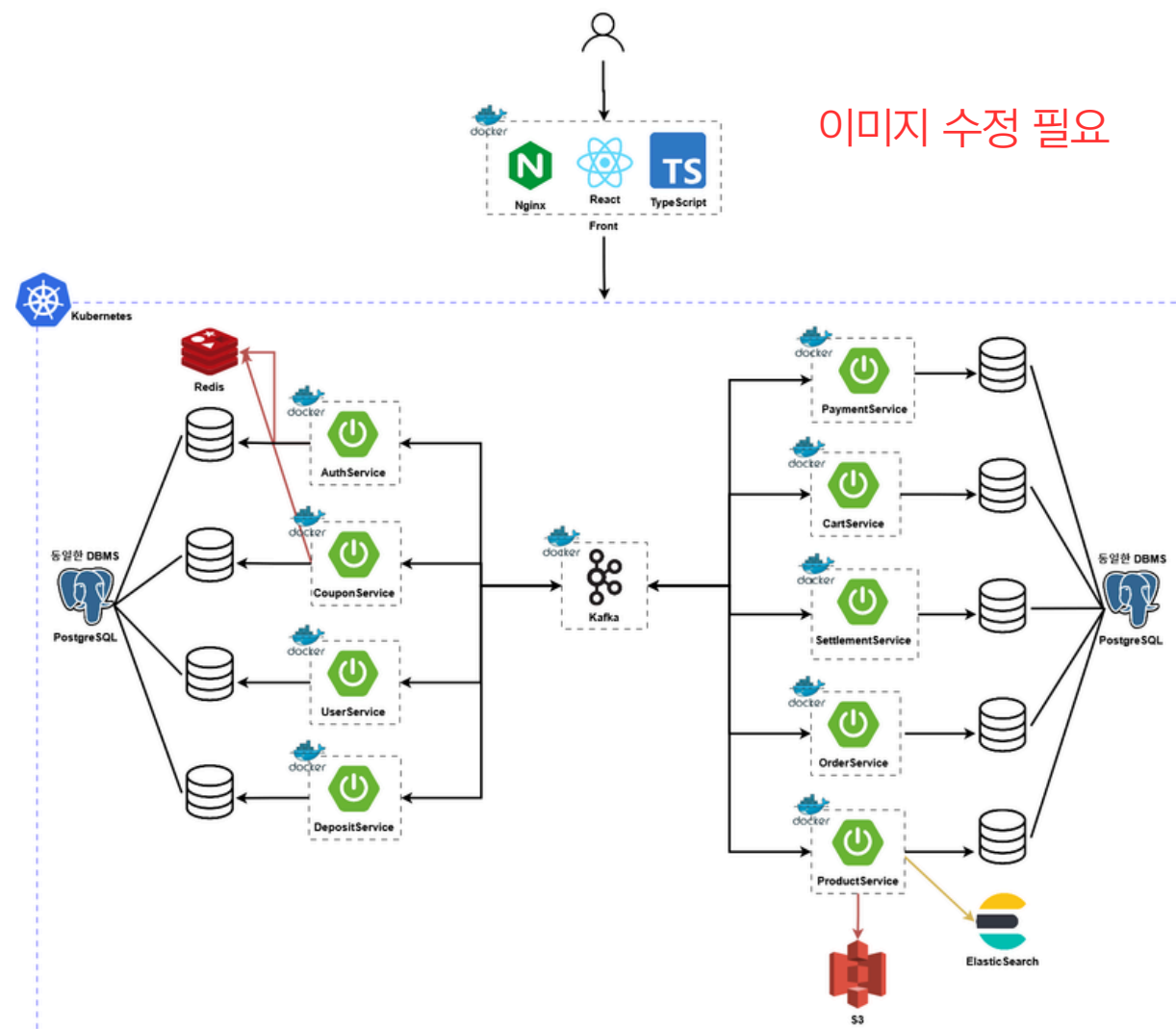
- Order는 단일 트랜잭션
- 도메인 간은 Kafka 기반 비동기 처리

설계 목적

- 결합도 감소 + 장애 격리 + 확장성 확보

이벤트 흐름





Modular Monolith → MSA 전환 전략

선택 의도

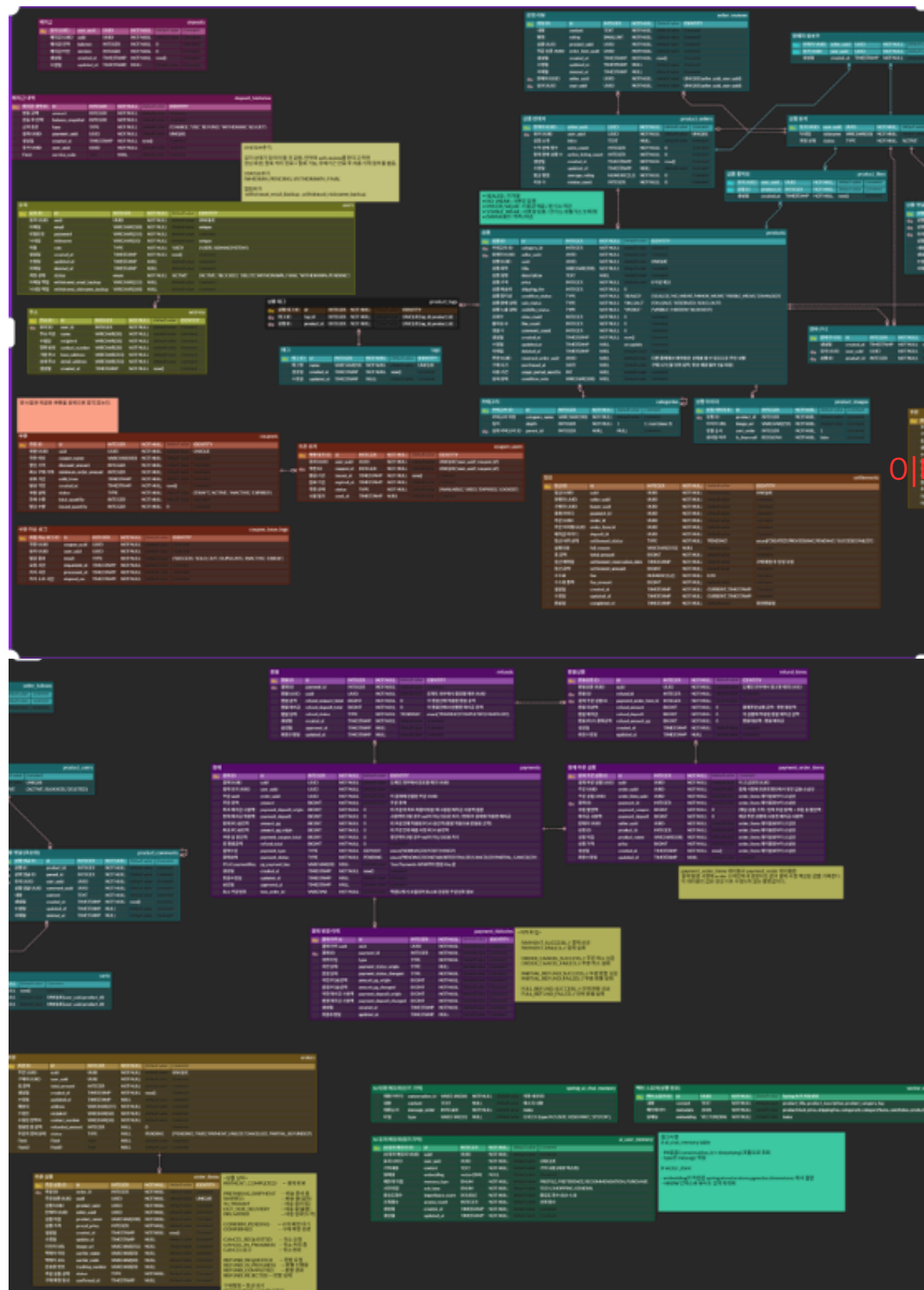
- 초기 개발 속도 확보
- 과도한 분산 복잡성 방지
- 추후 서비스 확장을 고려한 단계적 분리 전략

장점

- 초기 인프라/운영 부담 감소
- 단일 배포로 빠른 기능 검증 가능
- 도메인 경계가 명확하면 MSA로 자연스럽게 분리 가능

단점

- 단일 장애 지점(SPOF) 존재
- 트래픽 증가 시 확장 한계
- 모듈 간 의존성 관리 실패 시 분리 어려움



이미지 수정 필요

도메인 분리 기준 (유저 / 주문 / 결제 / 정산 등) 분리 기준

- 비즈니스 책임 단위
- 데이터 변경 주체
- 트랜잭션 경계

장점

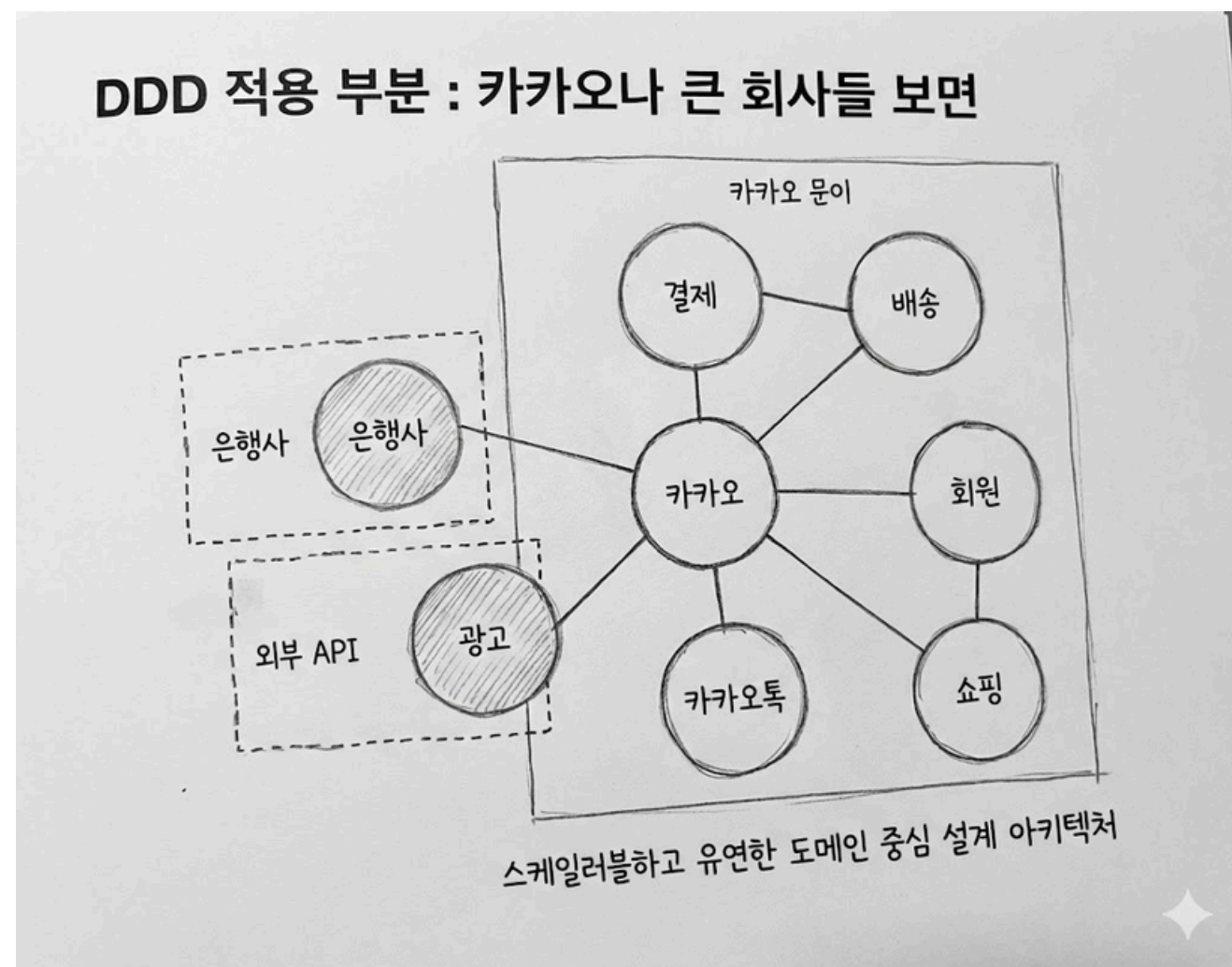
- 도메인 간 결합도 감소
- 팀 단위 병렬 개발 가능
- 장애 격리 및 확장 용이

단점

- 도메인 간 통신 복잡성 증가
- 분산 트랜잭션 정합성 문제 발생 가능
- 이벤트 설계 미흡 시 강결합 재발생

→ “기능 단위”가 아닌 “비즈니스 책임 단위”로 분리

이미지 수정 필요



DDD 적용 (Aggregate 단위 트랜잭션 중심 설계)

적용 방향

- Aggregate 내부에서만 강한 일관성 보장
- 도메인 서비스에 비즈니스 로직 집중
- 외부 도메인과는 이벤트 기반 통신

장점

- 트랜잭션 경계 명확화
- 도메인 로직 응집도 향상
- MSA 전환 시 자연스러운 서비스 단위 분리 가능

단점

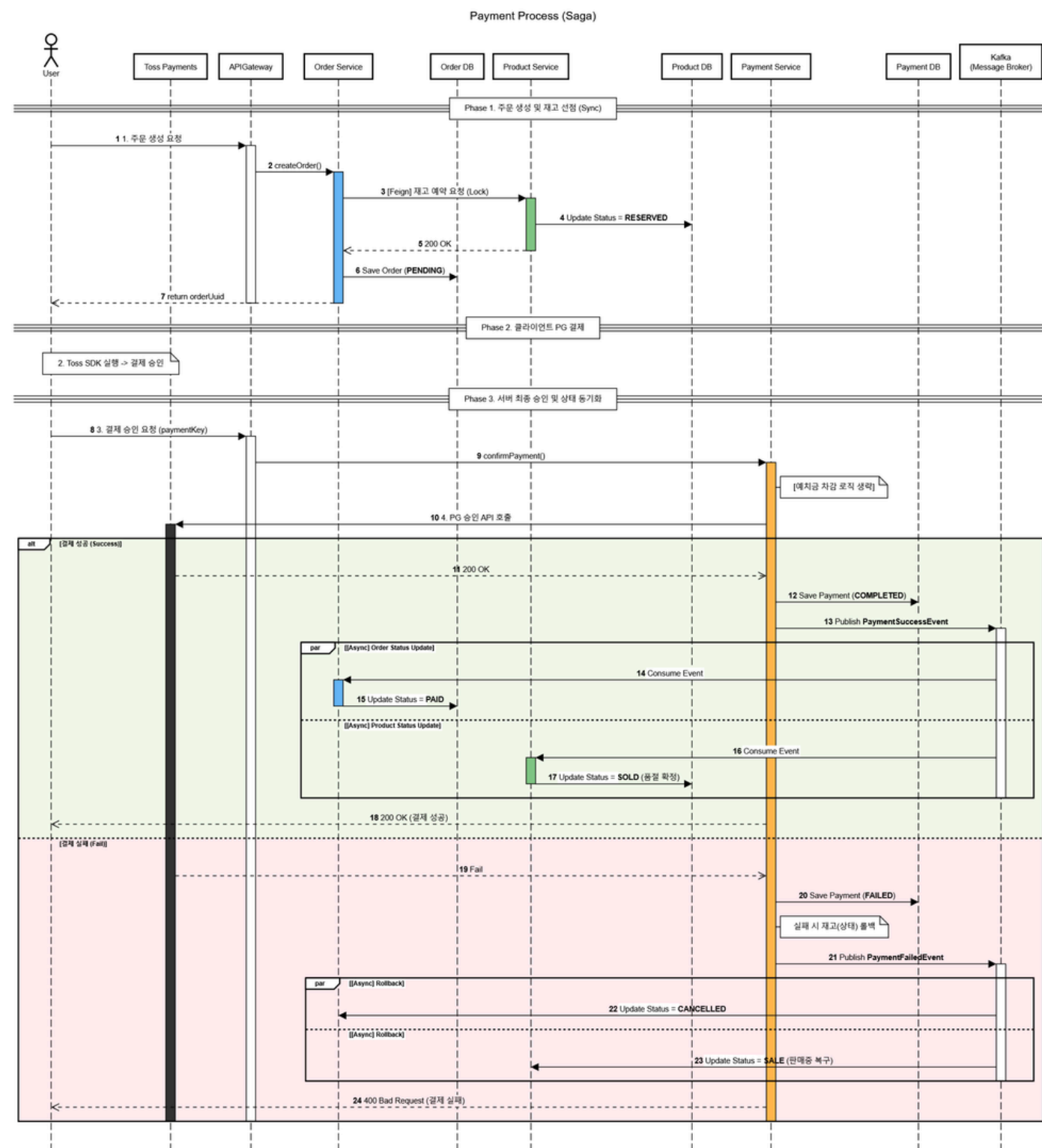
- 설계 난이도 증가
- 학습 곡선 높음
- 과도한 추상화 위험

아키텍처 설계 의도 장단점 작성하기

ular Monolith로 개발 속도 확보

II(유저/주문/결제/정산) 기준으로 추후 MSA 분리 가능하게 설계

: Aggregate 단위 트랜잭션, 도메인 서비스로 비즈니스 룰 집중



1. 초기 Spring Event로 시작 → Kafka로 전환 배경

기존 구조

- 초기에는 Spring Application Event 기반 동기 이벤트 처리
- 단일 애플리케이션 내 모듈 간 통신에 적합
- 하지만:
 - 서비스 분리 확장에 한계
 - 장애 격리 부족
 - 재시도/비동기 처리 제약

개선 방향

MSA 확장 가능 구조 + 장애 격리 + 확장성 확보 필요

➔ Kafka 기반 이벤트 브로커 구조로 전환

2. Saga 패턴 적용 이유

분산 트랜잭션 문제 해결

- 2PC (Two Phase Commit) 미사용
- 각 서비스는 로컬 트랜잭션만 관리

장애 격리

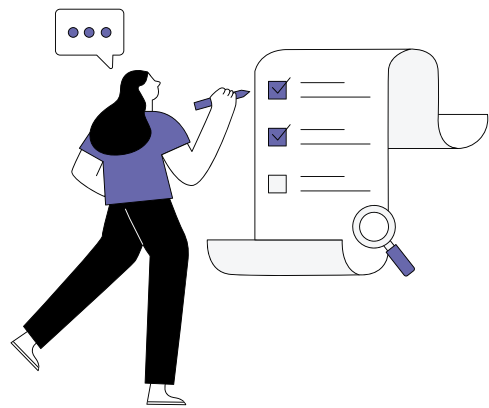
- 특정 서비스 장애가 전체 시스템을 멈추지 않음

확장성

- 새로운 서비스 추가 시 이벤트 구독만 추가하면 됨



Contents



01. 프로젝트 개요	02. 협업 및 개발 프로세스	03. MSA 아키텍처
04. DUKKU 시연 영상	05. 핵심 기능	06. 인프라 구축 및 시스템 고도화
07. 한계점 및 향후 개선 방향	08. 팀 구성과 역할	09. KPT 회고

Deposit 초기화 이벤트

- 실패 시 유저 롤백
- Saga 패턴 적용
- 예시 : 중복 클릭 처리

결제 프로세스 보상 처리 구조

- 주문 생성 시 예치금 초기화 이벤트 발행
- 결제 성공/실패 이벤트에 따라 주문 상태 전이
- 실패 시 사용자 포인트/예치금/주문 상태 보상 처리
- Saga 오케스트레이션으로 단계별 성공/실패를 추적

이 부분이 가장 중요함 (포트폴리오 핵심)

- MSA 단점이 트랜잭션 처리 복잡성 단점 설명
- MSA 설계 중의 트러블 슈팅 설명(원인, 해결, 성과 또는 느낀점) + 도메인마다 트러블 슈팅 작성
 - 예시 : 중복 결제, 외부 시스템 오류, 도메인간 트랜잭션 처리 어려움
 - 트러블 슈팅을 더 발견하기 위한 성능, 정합성 테스트 설명(더 deep 한 트러블 슈팅 문제해결 과정 보여주기)
 - 예시 : 점진적 요청을 통해, Kafka의 리밸런싱 이슈를 발생해서 멍등성 체크에 대해 예외 케이스들을 알 수 있었다.
- 개발은 끝이 아닌 운영 계획 및 전략은 어떻게 할 것이며, 대응은 어떻게 할 것인지 프로세스 정의(완벽하지 않아도 됨)

간단한 기능 설명+도입 이유

구현 핵심 포인트

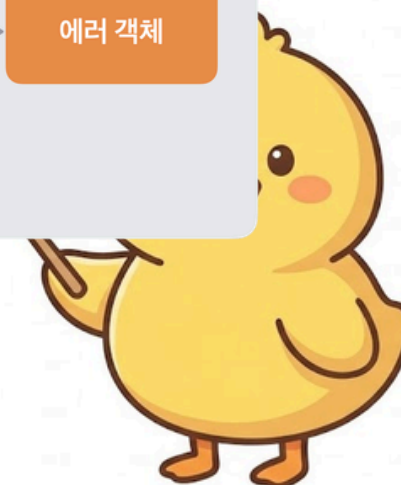
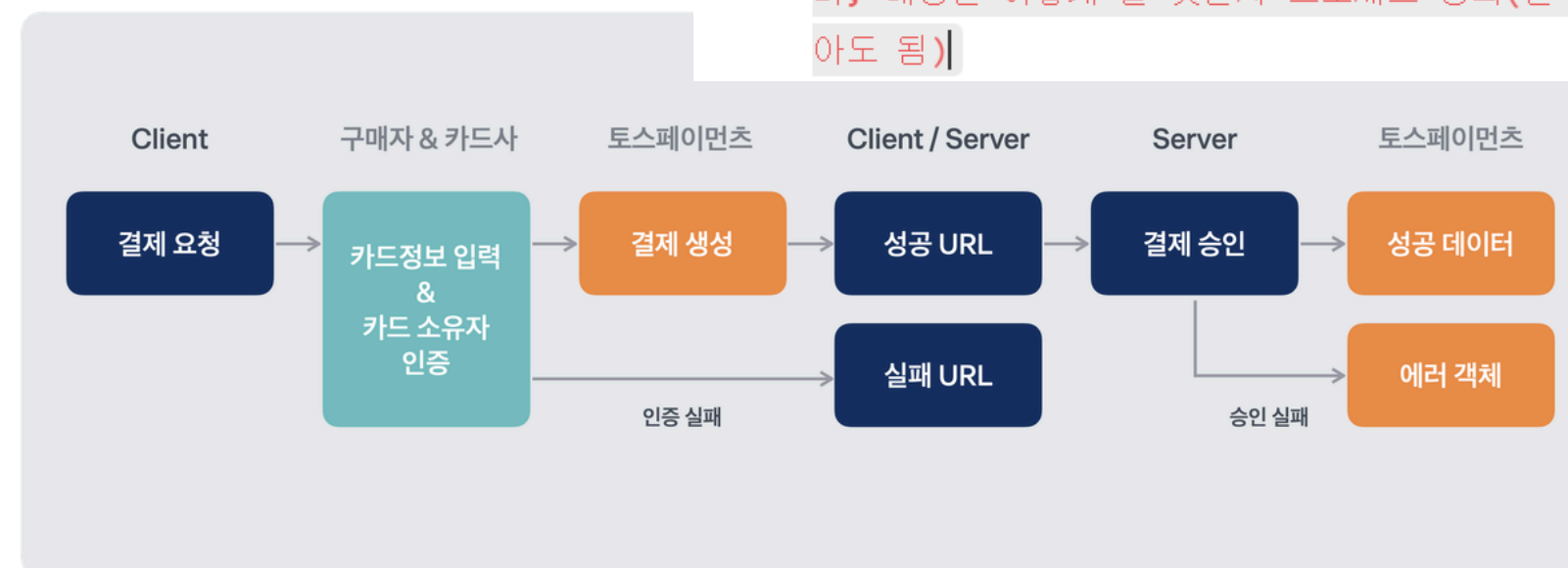
Saga 오케스트레이션으로 단계별 성공/실패를 추적
로그를 통하여 데이터 수집 등

예시 이미지 이벤트 흐름도

문제 상황

해결책 or 개선책

(개선 1,2,3차 까지 있으면 작성해도 좋음)



- 예시 : 중복 클릭 처리

문제 1. 보상 처리 중 DB 롤백 시 실패 이벤트까지 유실되는 문제

원인: 보상 로직 수행 중 트랜잭션이 롤백되면, 동일 트랜잭션 내에 발행된 실패 이벤트도 함께 롤백되어 외부로 전파되지 않는 현상 발생

해결:

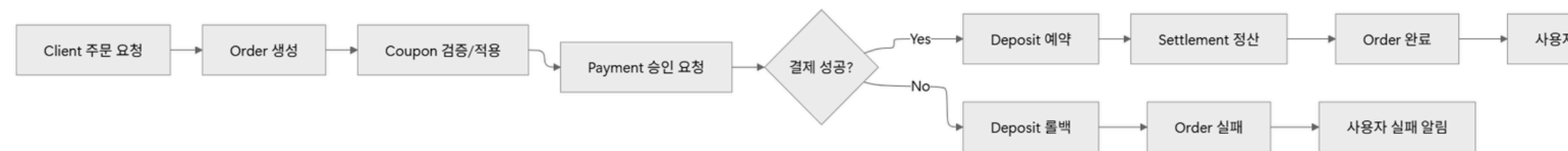
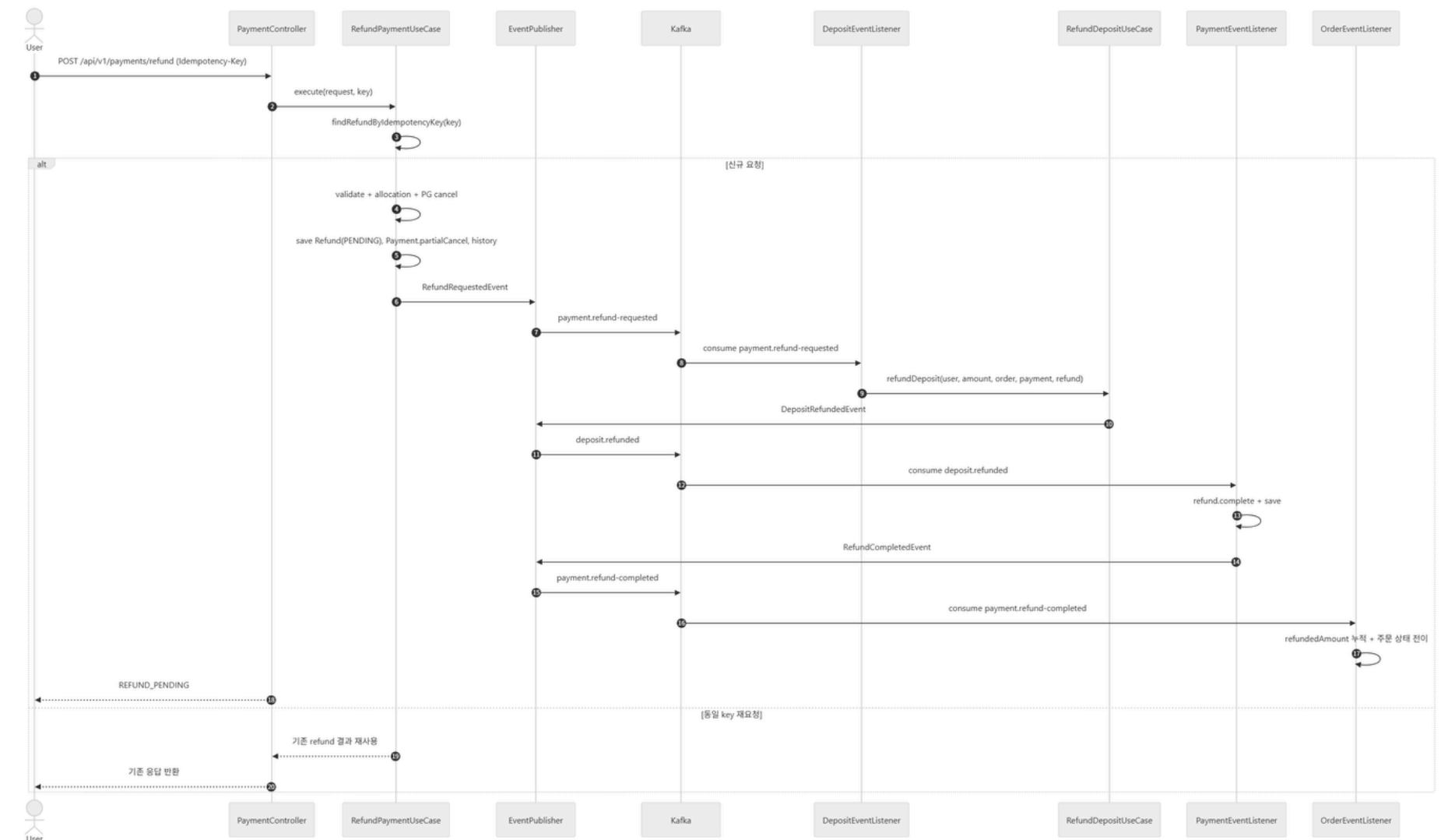
- 트랜잭션 완료 이후(afterCompletion) 시점에 이벤트를 발행하도록 구조 변경
 - DB 커밋/롤백 여부와 분리된 후행 이벤트 발행 방식 적용
- 보상 실패 상황에서도 이벤트 유실 없이 후속 복구 로직 정상 동작

문제 2. 중복 이벤트/재시도 시 상태가 여러 번 변경되는 문제

원인: Kafka 재처리 또는 중복 이벤트 수신 시 동일 결제 건의 상태가 반복 변경되는 현상 발생

해결:

- 결제 이력 기반 먹등 체크(hasHistoryType) 로직 추가
 - 처리 완료 이벤트를 별도 저장(processed_refund_events) 하여 중복 처리 방지
 - 상태 전이 전 현재 상태 검증 로직 강화
- 재처리/중복 수신 상황에서도 상태 일관성 유지



낙관적 락은 충돌 시 재시도, 비관적 락은 작업 전 충돌하지 않게 잠그고 시작

동시성 처리 개선

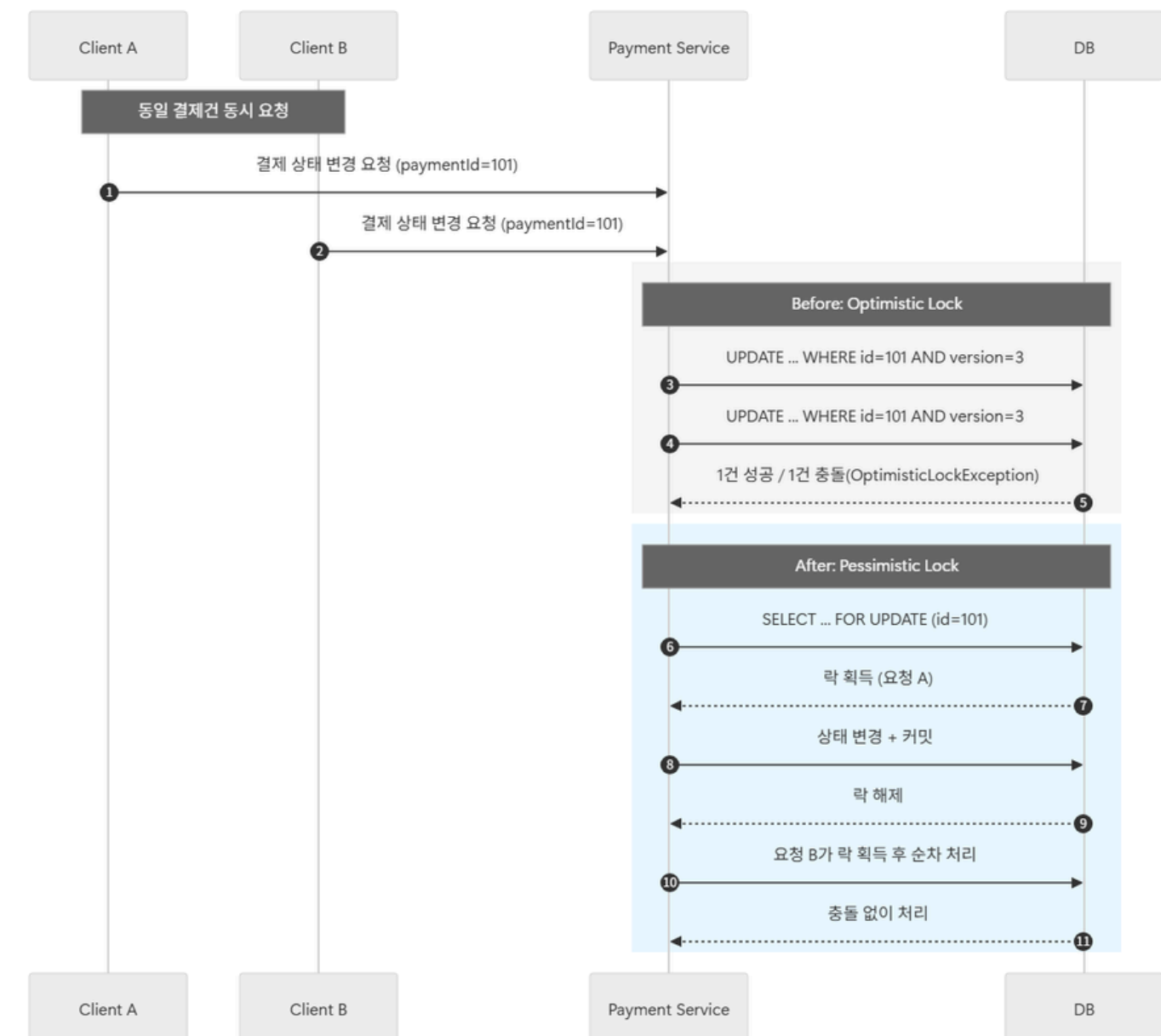
결제/환불 구간은 동시 요청이 빈번하게 발생하는 고경합 영역이었습니다.

기존 낙관적 락 방식에서 버전 충돌이 다수 발생하여 재시도 로직이 증가했고, 그로 인해 처리 지연과 실패율이 상승하는 문제가 발생했습니다.

이에 따라 핵심 갱신 구간에 비관적 락(PESSIMISTIC_WRITE)을 적용하여 트랜잭션 내 선점 락 기반의 단일 처리 구조로 개선했습니다.

구현 핵심 포인트

- 고경합 구간(결제/환불)에 PESSIMISTIC_WRITE 적용
- 트랜잭션 내 선점 락으로 단일 처리 보장
- 동일 결제 건의 중복 상태 변경 방지
- 충돌 재시도 감소 → 처리 안정성 향상



Deposit 초기화 이벤트

- 실패 시 유저 롤백
- aga 패턴 적용
- 시 : 중복 클릭 처리

문제. 동시성 이슈

- 시스템 지갑을 두고 거래 금액을 중간에서 관리하는 방식으로 거래 구현
- 시스템 지갑이 하나기 때문에 거래가 몰릴 경우 동시에 계좌에 접근 상황 자주 발생
- 동일 결제/환불 건에 동시 요청이 몰릴 때 반복적인 재시도 발생으로 지연 발생
- 유저가 실패에 대하여 인지하기 어려움 (장애 상황으로 인지)

원인 분석

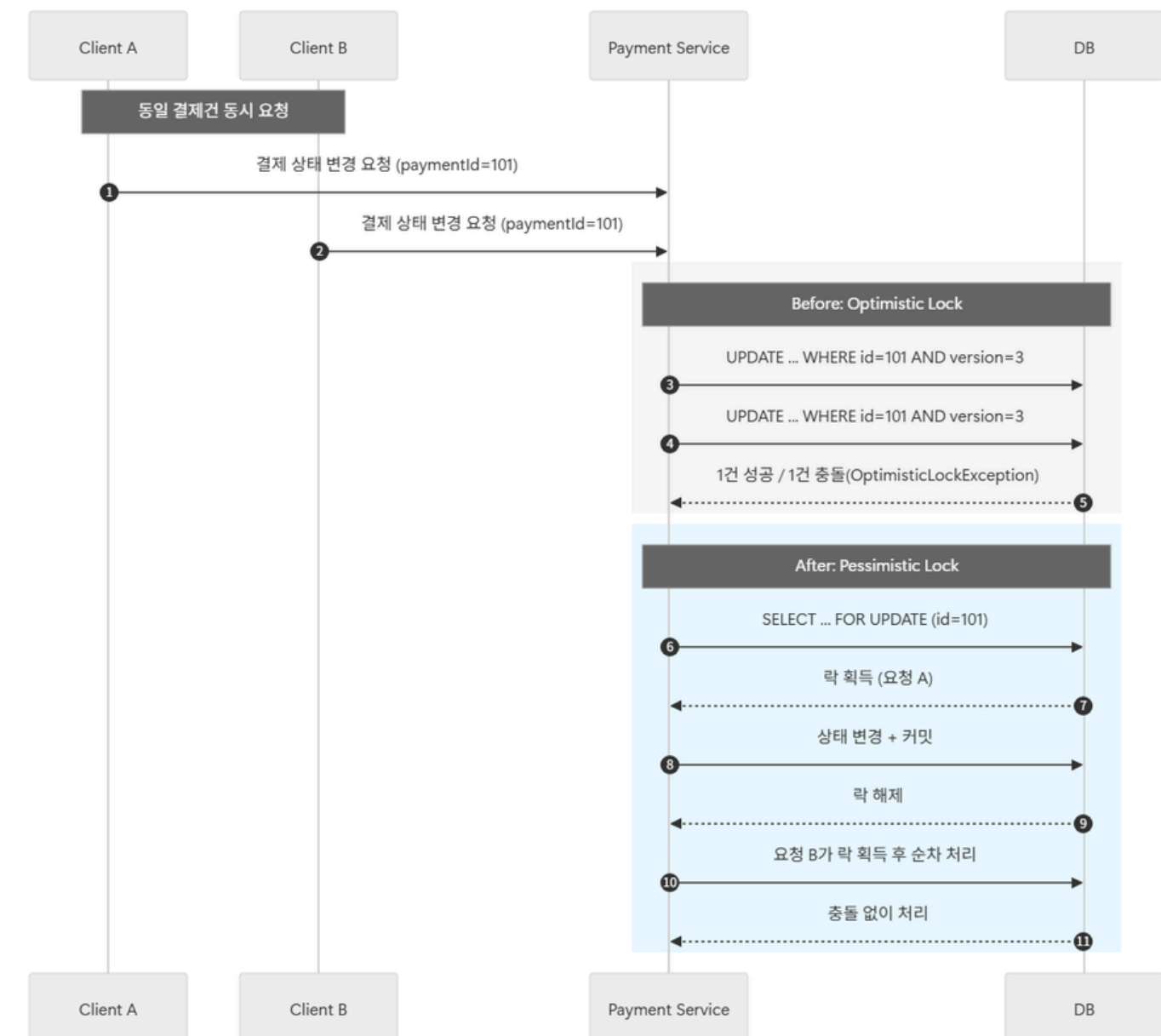
- 낙관적 락 기반 거래가 몰릴 경우 버전 충돌이 빈번하게 발생
- 충돌 후 재시도 로직이 트래픽 피크에서 추가 부하를 유발

해결 전략

- 핵심 상태 변경 구간을 비관적 락(PESSIMISTIC_WRITE)으로 전환
- 트랜잭션 내 선점 락으로 동일 엔티티의 동시 갱신을 직렬화
- 락 범위를 최소화해 전체 처리량 저하를 방지

검증 및 결과

- 동일 키 동시 요청 부하 테스트로 충돌/실패 케이스 재현
- 락 전환 후 버전 충돌 예외가 제거되고 상태 불일치가 해소됨
- 재시도 횟수 감소로 응답 안정성이 개선됨



도입 배경

결제 도메인은 돈이 직접적으로 오가는 영역이기 때문에 금액의 정합성이 매우 중요
 kafka는 기본적으로 이슈 상황에서 메시지가 유실될 가능성에 대비한 재처리 로직이 존재
 고부하 상황에서 메시지가 유실되거나 성공 응답을 못 받으면 같은 메시지를 2번 이상 받을 가능성

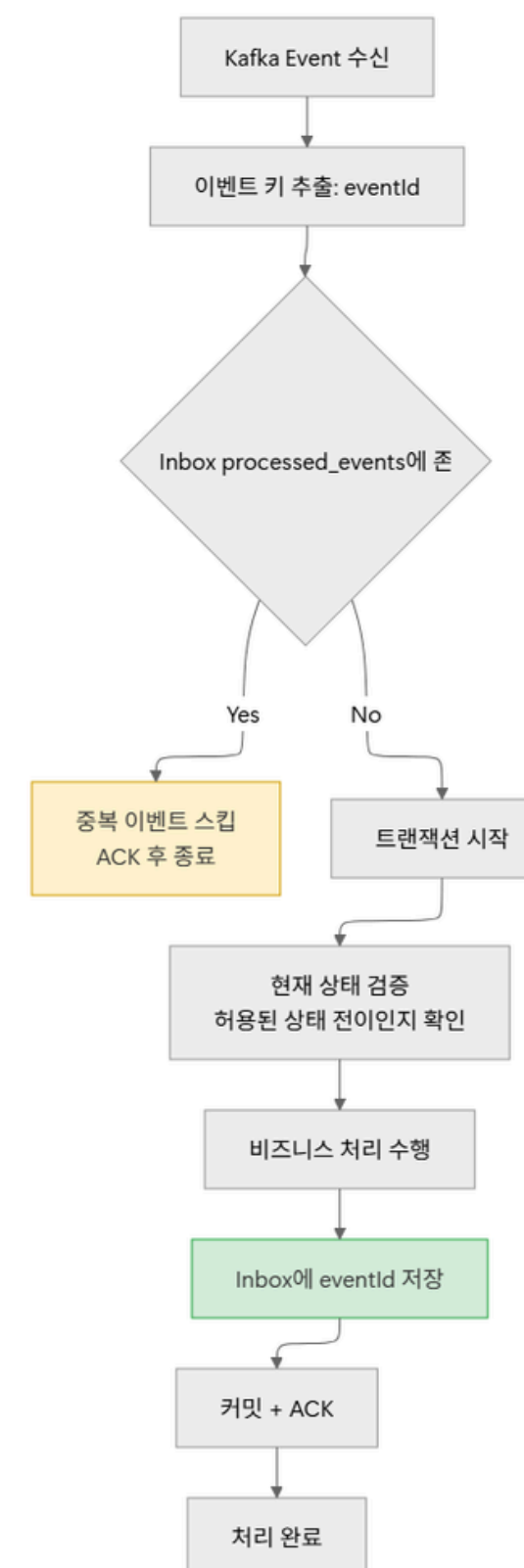
멱등성 처리

Kafka 기반 비동기 구조에서는 재전송 및 중복 수신 가능성을 항상 고려할 필요
 이미 받은 이벤트 목록을 저장해뒀다 비교하는 Inbox 패턴,
 발행할 메시지를 저장해뒀다 발행 성공한 이벤트를 완료처리하는 Outbox 패턴

구현 복잡도를 고려해 Outbox 패턴은 도입하지 않았으며,
 대신 소비자 측 Inbox 테이블을 활용한 멱등성 보장 전략 채택
 이벤트 고유 키를 기준으로 처리 이력을 저장하고, 이미 처리된 이벤트는 즉시 스킵하도록 설계

구현 핵심 포인트

- processed_events(Inbox) 테이블로 처리 이력 관리
- 이벤트 고유 키 기준 중복 차단
- 비즈니스 상태 전이 전 현재 상태 검증 로직 추가
- Kafka 재전송/중복 수신 상황에서도 중복 반영 방지
- 재처리 상황에서도 상태 일관성 유지



SAGA 패턴 기반 보상 처리

분산 트랜잭션 정합성 설계

결제 이후 예치금, 주문, 쿠폰 등

다중 도메인이 연계되는 구조에서 일부 단계 실패 시

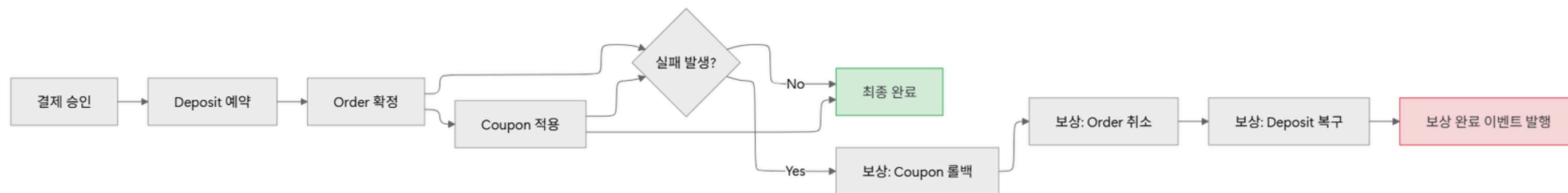
전체 데이터 정합성이 깨질 위험이 존재했습니다.

이를 해결하기 위해 2PC 대신

SAGA 패턴 기반 단계별 이벤트 처리 및 보상 트랜잭션 구조를 적용했습니다.

구현 핵심 포인트

- 이벤트의 단계별 흐름 체인 설계
- 실패 시 재시도, 지속적 실패 시 critical 로그 기록 후 단계적 롤백
- 도메인별로 분리된 각 단계에 맞춰 각각 트랜잭션 롤백 수행
- 사전 정의된 보상 액션이 역순으로 실행
- 후속 이벤트를 발행하여 보상 트랜잭션의 다음 단계로 연결
- 부분 실패에 대한 자동 복구 가능
- 도메인 간 결합도 감소 및 장애 격리 강화



문제

- 결제 후 예치금/주문/쿠폰 등 후속 단계 중 일부 실패 시 전체 정합성 깨짐
- 단일 트랜잭션으로 묶기 어려워 부분 성공 상태가 남을 수 있음

원인 분석

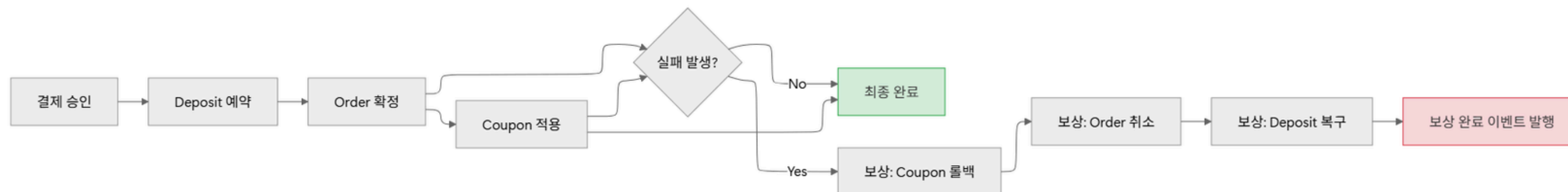
- 실패 이벤트가 트랜잭션 경계에 묶이면 복구 트리거 유실 가능성 존재
- 분리된 모듈은 하나의 트랜잭션으로 묶을 수 없어 후속 조치가 필요

해결 전략

- 실패 시 사전 정의된 보상 트랜잭션을 역순 실행해 상태 복구
- Kafka 등의 메시지 큐 시스템을 통해 다음 도메인에서의 후속 동작을 정의하는 이벤트를 호출

검증 및 결과

- 단계별 강제 실패(예치금/주문/쿠폰) 주입 테스트 수행
- 실패 케이스에서 보상 플로우가 정상 동작해 최종 정합성 회복 확인
- 부분 장애 발생 시에도 서비스가 복구 가능한 구조로 안정성 향상



인증 및 보안 설계

저희 서비스는 MSA 구조이기 때문에 서버 간 인증 상태를 공유하지 않는 Stateless 방식이 필요했습니다.

따라서 세션 기반 인증 대신 JWT 기반 인증 구조를 설계했습니다.

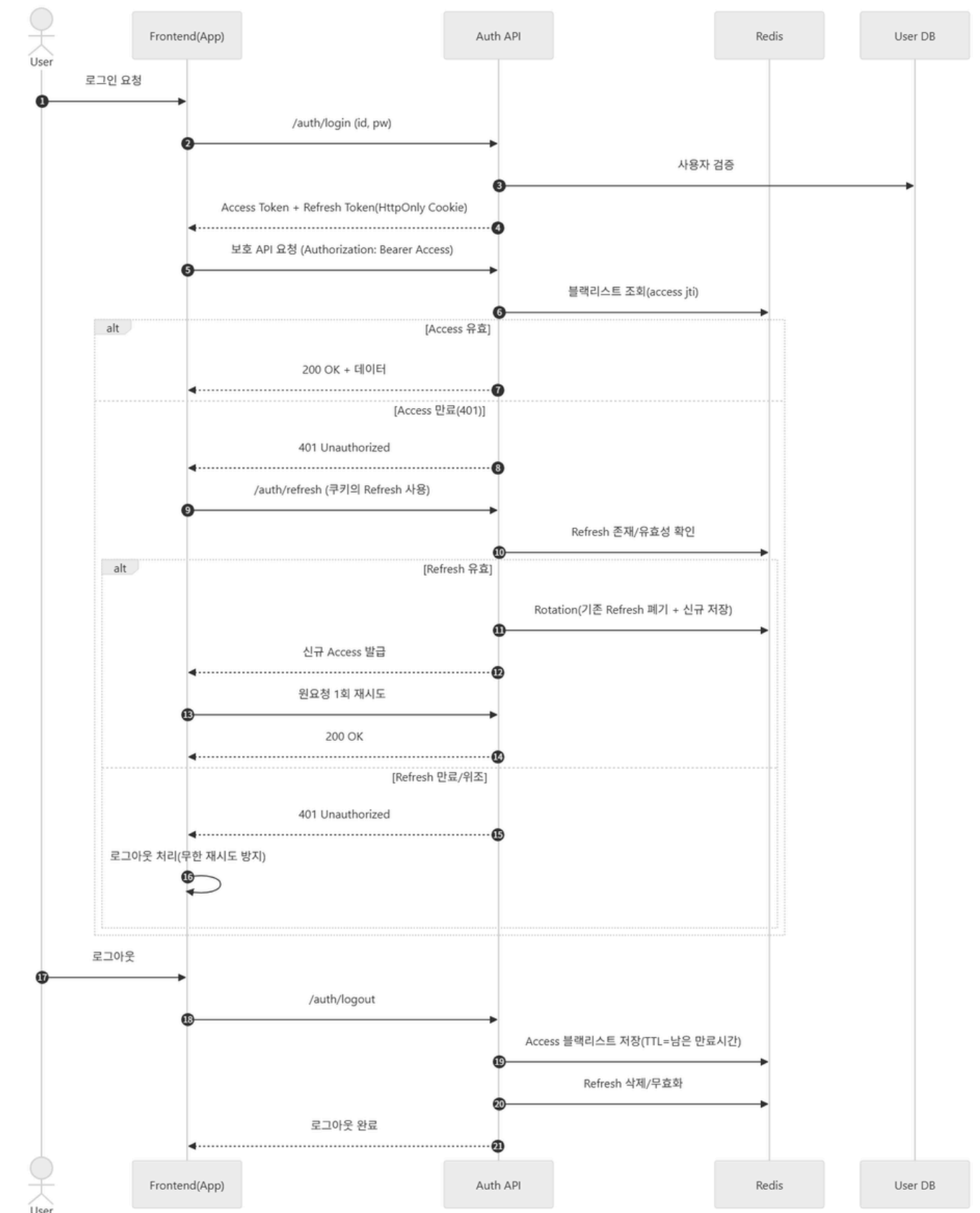
Access Token + Refresh Token 전략을 사용했고, 보안 강화를 위해 Refresh Rotation과 Redis 기반 블랙리스트 전략을 적용했습니다

구현 핵심 포인트

- Auth-User 분리 구조(검증/발급 책임 분리)
- JWT + Redis 기반 Refresh 검증
- Sliding + Absolute 만료 정책
- 회원가입 중복 방지(Idempotency-Key + 분산락)
- 제재 계정 로그인 차단
- Redis 기반 RefreshToken 저장/검증

간단한 기능 설명을 굳이 해야할까 싶음 어차피 다들 아는 기능일 텐데 그러면 왜 선택했는지를 포트폴리오 관점에서 설명하는게 더 좋아보임

예시 이미지



문제 1. 상황 회원가입 중복 요청으로 계정 충돌 발생

- 회원가입 API 재시도 / 중복 클릭
- 동시성 환경에서 계정 중복 생성 시도
- 단순 DB unique 제약만으로는 사용자 경험 불안정

원인 분석

- 네트워크 지연 상황에서 재요청 발생
- 트랜잭션 경합 시 에러 응답 반환
- 클라이언트는 성공 여부를 알 수 없음

해결 전략

- Idempotency-Key를 Redis에 저장
- IN_PROGRESS / COMPLETED 상태 관리
- 요청 해시 기반 동일 요청 검증
- 이메일 단위 분산락 적용 (동일 가입 직렬화)

검증 및 결과

- 동시 요청 테스트 수행
- 중복 가입 재현 테스트 통과
- 동일 키 재요청 시 동일 응답 보장
- 사용자 경험 안정화



문제 2. Refresh Token 탈취 및 재사용 공격 위험

- Refresh Token 탈취 시 무한 세션 연장 가능성
- JWT 자체 검증만으로는 서버 측 “유효한 토큰” 여부 판단 불가

원인 분석

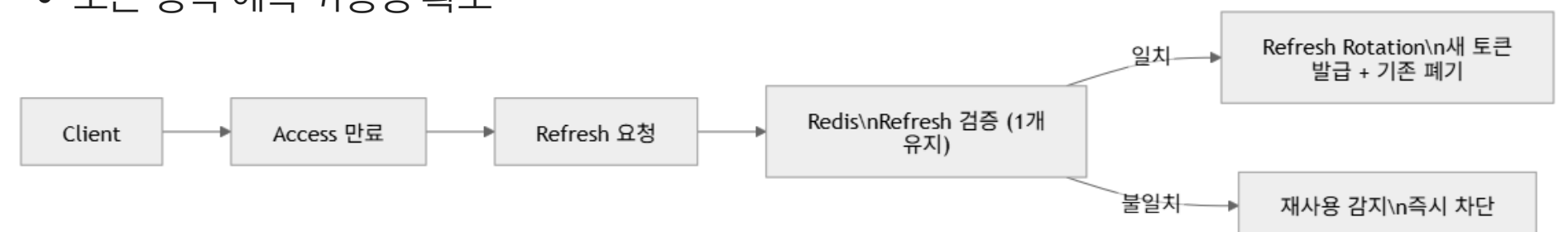
- JWT는 stateless
- 탈취 후 재사용 공격 방어 한계 존재

해결 전략

- 사용자당 Refresh Token 1개만 Redis에 저장
- 불일치 시 즉시 무효화
- Refresh Rotation 도입
- 매 갱신 시 기존 토큰 폐기
- Absolute Expiration 정책 도입
- 회전해도 최대 수명 제한

검증 및 결과

- 탈취 토큰 재사용 시나리오 테스트
- 재사용 시 즉시 차단 확인
- 장기 세션 누수 방지
- 토큰 정책 예측 가능성 확보



이 부분에 추후 개선방안 까지 작성을 할지 너무 두머치인거 같기도

API-Gateway 추가

하이브리드 검색 적용

AI 모듈의 상품 검색에 키워드 검색(PGroonga)과 벡터 유사도 검색(pgvector)을 결합한 하이브리드 검색을 적용했다. 두 검색 결과를 RRF(Reciprocal Rank Fusion) 알고리즘으로 합산하여 최종 순위를 결정한다.

Spring AI의 VectorStore(vector_store 테이블)는 벡터 유사도 검색만 지원하여, 하이브리드 검색을 위해 PGroonga 전문 검색과 pgvector 코사인 유사도 검색 그리고 metadata 필터링을 단일 SQL로 처리하여, PGroonga + pgvector + metadata 검색을 적용하여 검색 성능을 향상시켰다.

구현 핵심 포인트

- PGroonga + pgvector + metadata 하이브리드 검색
- RRF(Reciprocal Rank Fusion) 알고리즘 적용

검색 유형	잘 되는 경우	못 하는 경우
키워드 (PGroonga)	"스노우피크 텐트" — 정확한 상품명/브랜드명	"캠핑 갈 때 쓸만한 가벼운 것" — 의미 검색 불가
벡터 (pgvector)	"캠핑 갈 때 쓸만한 가벼운 의자" — 의미적 유사도	"스노우피크" — 정확한 단어 매칭이 보장 안 됨
하이브리드 (RRF)	양쪽 모두 커버	—

문제 1. 벡터 유사도 검색시 특정 상황에서 검색 성능 저조

- 키워드(상품명, 모델명, 브랜드) 검색:
 - 의미적으로 유사하더라도 키워드가 약간 다르거나 오타/다양한 표현을 가진 문서를 찾지 못함.
- 벡터 유사도 검색:
 - 벡터 임베딩 기반 검색은 의미적 유사도를 기준으로 상위 결과를 반환하므로 정확 일치 항목을 놓칠 수 있음.

원인 분석

- 임베딩의 일반화 특성
 - 임베딩은 의미적 유사성에 초점. “Apple iPhone 13 Pro”와 같이 정밀한 토큰/명칭이 중요한 경우, 의미상 유사한 다른 문서들이 우선될 수 있음.
- 단일 검색 방식 의존
 - 하나의 방법에만 의존하면 그 방법의 실패 모드(예: lexical miss / semantic drift)에 그대로 취약.
- 메타데이터(필터) 미사용
 - 카테고리/브랜드/가격대 같은 구조화된 필터를 함께 쓰지 않으면 유사 문서들이 섞여 들어옴.

해결 전략

- PGroonga + pgvector + metadata 하이브리드 검색

검증 및 결과

- 정답 상품에 대한 유사도 비교 테스트 (단일 키워드 vs 단일 벡터 vs 하이브리드)

문제 2. 키워드 검색 유사도와 벡터 검색 유사도는 값의 스케일(범위)과 의미가 전혀 다름

- 키워드 검색 유사도(예: BM25, tf-idf, ts_rank) 와 벡터 검색 유사도(예: cosine similarity, inner product) 는 값의 스케일(범위)과 의미가 전혀 다르기 때문에 단순히 두 점수를 합산하면 한쪽이 압도적으로 지배하거나, 가중치 튜닝이 불가능해지는 문제가 생김. 단순히 두 점수를 합산하면 한쪽이 압도적으로 지배하거나, 가중치 튜닝이 불가능해지는 문제가 생김.

원인 분석

- 키워드 검색 유사도와 벡터 검색 유사도의 값의 스케일 범위와 의미가 다름

해결 전략

- 두 검색기 점수를 직접 비교하지 말고, 각 검색기에서의 “순위(rank)”만 이용해 합치는 순위 기반 융합 (RRF, Reciprocal Rank Fusion)을 사용

검증 및 결과

- A/B 테스트

RRF (Reciprocal Rank Fusion) 알고리즘

공식: $RRF_score(d) = 1/(k + rank_keyword(d)) + 1/(k + rank_vector(d))$

- k = 60 (smoothing constant, AiSimilarityPolicy.RRF_K)
- rank_keyword(d): PGroonga 검색 결과에서의 순위 (1부터 시작)
- rank_vector(d): pgvector 코사인 유사도 기준 순위 (1부터 시작)
- 한쪽에만 등장한 문서: 해당 쪽 점수만 반영 (FULL OUTER JOIN)

"스노우피크 텐트"로 검색 시:

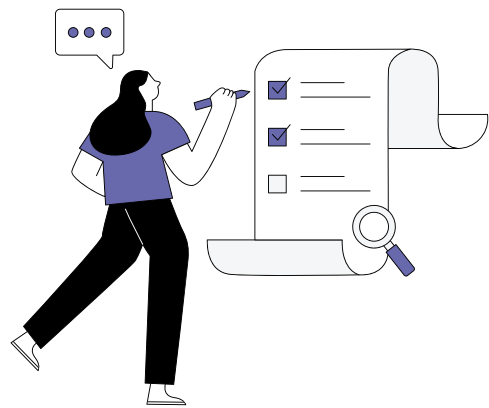
상품	keyword rank	vector rank	RRF score
스노우피크 랜드록 텐트	1	3	$1/61 + 1/63 = 0.0323$
힐레베르그 아틀라스 4인용	—	2	$0 + 1/62 = 0.0161$
스노우피크 티타늄 싱글머그	2	—	$1/62 + 0 = 0.0161$

키워드 정확 매칭 + 벡터 유사도 양쪽에 모두 등장하는 "스노우피크 랜드록 텐트"가 최고 점수를 받는다.

k=60의 의미

- k가 작으면 순위 차이에 민감 → 1위와 2위 점수 차이가 큼
- k가 크면 순위 차이에 둔감 → 양쪽 검색에 모두 등장하는 것이 중요
- k=60은 학계와 실무에서 널리 쓰이는 표준값

Contents

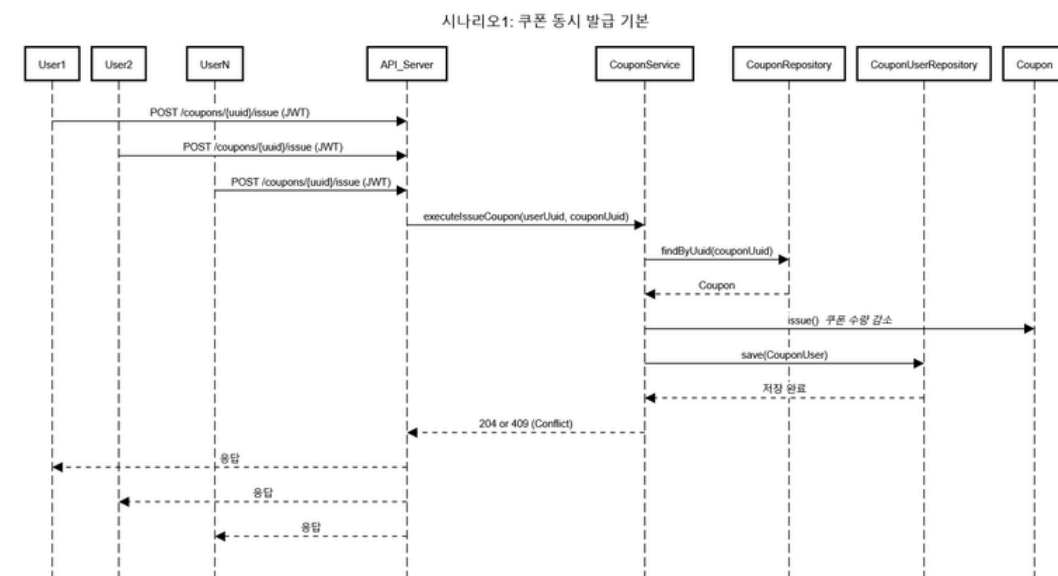


01. 프로젝트 개요	02. 협업 및 개발 프로세스	03. MSA 아키텍처
04. DUKKU 시연 영상	05. 핵심 기능	06. 인프라 구축 및 시스템 고도화
07. 한계점 및 향후 개선 방향	08. 팀 구성과 역할	09. KPT 회고

시나리오1(동시성 제어가 없는 초기 모델)

1. 테스트 목적

본 테스트는 쿠폰 발급 시스템의 초기 설계가 대규모 동시성 요청(Traffic Spike) 환경에서 **데이터 무결성(Integrity)**과 **비즈니스 정합성(Consistency)**을 유지할 수 있는지 검증하는 데 목적이 있다.



추후 운영 계획(운영하지 않더라도) 넣으면 좋을 것 같은데
내용이 좀 많이 길어질 것 같아서 고민입니다

한 슬라이드에 넣기에 내용이 많아서 나눠서 넣는게 나을 것 같음
이 부분 찬희님 쿠폰테스트 내용 그대로 넣어도 좋음
시나리오 별로 내용도 있어서 개선 1,2,3 차 아주 굳

2. 테스트 결과 통계 (5회 반복)

모든 테스트에서 시스템은 1,000건의 요청을 모두 '성공'으로 처리했으며, 실제 DB 상태는 심각한 불일치를 보였다.

지표	1차	2차	3차	4차	5차
HTTP 성공 응답	1,000건	1,000건	1,000건	1,000건	1,000건
쿠폰 총 수량	100개	100개	100개	100개	100개
DB: coupon	1,000건	1,000건	1,000건	1,000건	1,000건
DB: issued	43	37	53	47	47

3. 기술적 결론 및 시사점

- 사용자 기만적 응답: 시스템은 1,000명에게 성공했다고 응답했으며, 실제 비즈니스 규칙(100개 제한)은 완전히 파괴되었다.
- 데이터 불일치의 극치: 발급 내역(1,000건)과 잔여 수량 카운트(47)가 전혀 맞지 않는 '데이터 정합성 붕괴' 상태가 발생했다.
- 격리 수준의 한계: 기본 격리 수준(Read Committed)은 이러한 '두 번째 수정 분실 문제'를 막아주지 못하며, 원자적 연산이나 별도의 락(Lock) 메커니즘이 필수적임을 시사한다.

부하 테스트 및 성능 개선

- k6 시나리오: 로그인, 상품조회, 주문생성, 결제요청
- 병목: DB 인덱스 미흡, N+1, 동기 처리 과다
- 개선: 인덱싱, fetch 전략 조정, 비동기 이벤트 전환
- 결과: 응답시간/처리량/에러율 지표 개선(수치 삽입 권장)

1. 부하 테스트 및 성능 개선

문생성, 결제요청
처리 과다

- 개선: 인덱싱, fetch 전략 조정, 비동기 이벤트 전환
- 결과: 응답시간/처리량/에러율 지표 개선(수치 삽입 권장)

한 슬라이드에 넣기에 내용이 많아서 나눠서 넣는게 나을 것 같음
이 부분 찬희님 쿠폰테스트 내용 그대로 넣어도 좋음

시나리오 별로 내용도 있어서 개선 1,2,3 차 아주 굳

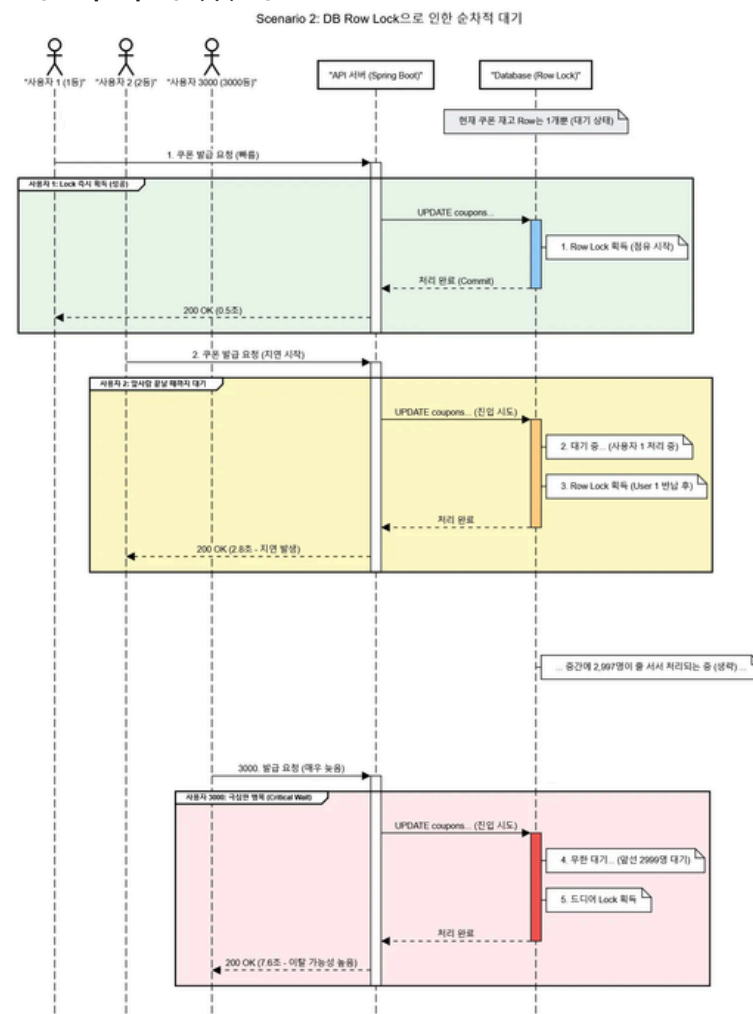
3. 기술적 결론

- 목표 달성 (정합성 & 가용성): 시나리오 1의 데이터 붕괴 문제를 해결했으며, 5,000명 규모의 고부하 상황에서도 서버 다운 없이 **에러율 0%**와 정확한 재고 차감을 달성함.
- 구조적 한계 (Physical Limitation): 디스크 기반 RDB의 I/O와 Lock 메커니즘으로는 3,000 VUs 이상의 대규모 트래픽에서 병목이 발생하며 처리량(TPS)이 저하됨.
- 해결책: 5,000명 이상의 대규모 트래픽을 지연 없이 처리하고 TPS를 확보하기 위해서는, 디스크가 아닌 In-Memory에서 원자적 연산을 수행하는 Redis 도입이 효율적임.

시나리오2(DB 원자적 연산을 통한 정합성 확보)

1. 테스트 목적

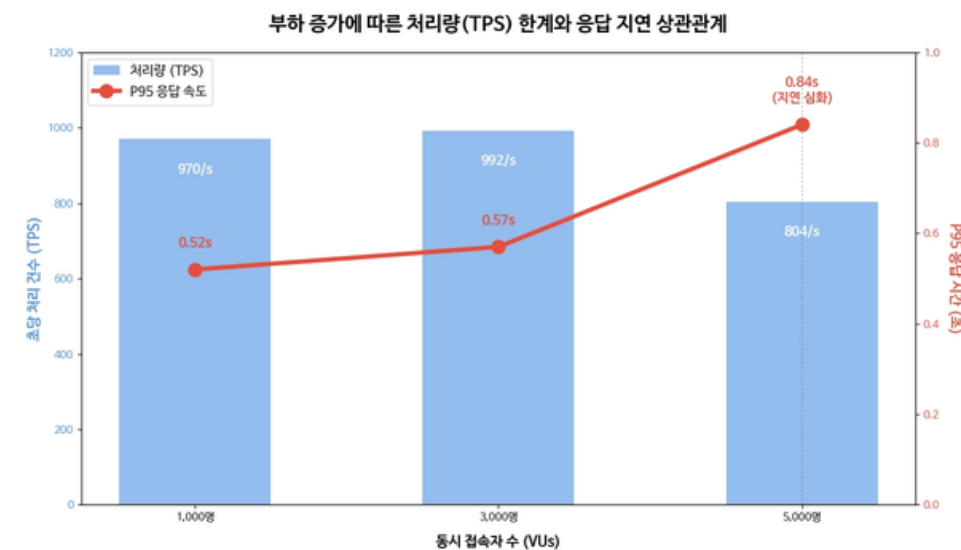
본 테스트는 데이터 정합성 해결을 위해 DB 원자적 연산 (Atomic Operation & Row Lock) 방식을 적용한 설계가, 트래픽 규모(1k ~ 5k VUs) 증가에 따라 **데이터 무결성 (Integrity)**을 유지하면서 **어느 수준의 성능 (Latency)**을 보장할 수 있는지 구조적 한계를 파악하는데 목적이 있다.



2. 부하 스케일별 테스트 결과 비교

부하가 증가함에 따라 데이터 정합성과 시스템 가용성(에러율 0%)은 완벽하게 유지되었으나, 응답 대기 시간 (Latency)은 소폭 상승하고 처리량(TPS)은 5,000 VU 구간에서 감소하는 병목 현상을 보였다

지표	1,000	3,000	5,000	비고
발급 성	100건	100건	100건	정합성
재고 소	900건	2,900건	4,900건	정상 로
요청 실	0건	0건	0건	가용성
평균 응	0.21초	0.32초	0.45초	부하 비
최악 응	0.52초	0.57초	0.84초	1초 미만
처리량	약	약	약	5k에서



1. 부하 테스트 및 성능 개선

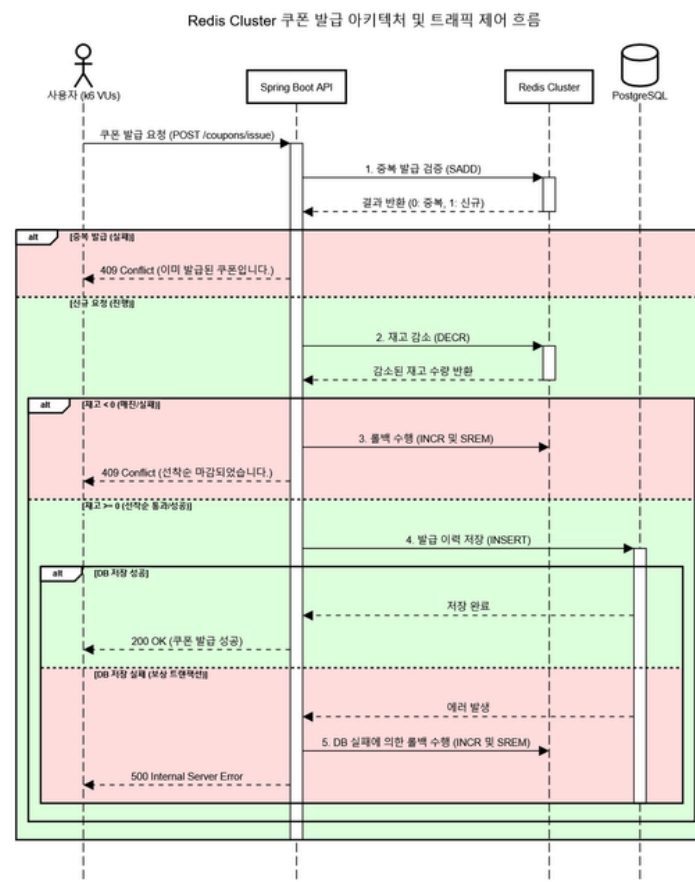
문생성, 결제요청
처리 과다

- 개선: 인덱싱, fetch 전략 조정, 비동기 이벤트 전환
- 결과: 응답시간/처리량/에러율 지표 개선(수치 삽입 권장)

시나리오3(Redis 도입 및 Cluster 전환을 통한 트래픽 제어)

1. 테스트 목적

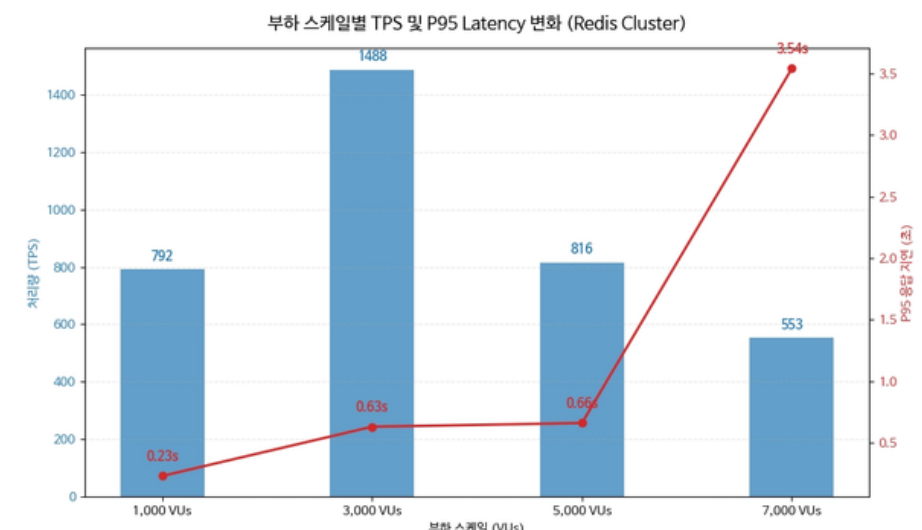
본 테스트는 기존 DB Row Lock 방식(시나리오 2)의 성능 저하 문제를 해결하기 위해 Redis를 도입하여 인 메모리 기반의 트래픽 제어(Traffic Throttling) 구조를 검증하는 데 목적이 있다. 특히 단일 Redis 인스턴스 (Lua Script 기반)의 성능 한계를 파악하고, Redis Cluster를 통해 대규모 트래픽 환경(1k ~ 7k VUs)에서 **데이터 무결성을 유지하며 처리량(TPS)과 응답 지연(Latency) **이 어떻게 개선되는지 분석한다.



2. 부하 스케일별 테스트 결과 비교 (Redis Cluster 기준)

부하 증가에 따라 DB 커넥션 점유 없이 탈락 요청을 409 Conflict로 즉시 반환하여 시스템 가용성(에러율 0%)을 100% 유지했다. 5,000 VUs까지 P95 응답 속도 1초 미만 을 방어했으나, 7,000 VUs 구간에서 응답 지연이 급증했 다.

지표	1,000	3,000	5,000	7,000	비고
발급 성	100건	100건	100건	100건	정합성
재고 소	900건	2,900	4,900	6,900	Redis
요청 실	0건	0건	0건	0건	에러 발
평균 응	0.15초	0.47초	0.28초	2.16초	-
최악 응	0.23초	0.63초	0.66초	3.54초	5k 부
처리량	약	약	약	약	3k 최



3. 기술적 결론

- 목표 달성 (정합성 & DB 보호): Redis를 활용한 트래픽 필터링을 통해 5,000명 이상의 고부하 상황에서도 DB 커넥션 고갈 없이 시스템 가용성을 100% 확보함.
- 성능 최적화: Lua Script 기반의 단일 스레드 병목을 분산 아키텍처(Cluster)로 개선하여 5,000 VUs까지 P95 1초 이내 응답을 달성함.
- 구조적 한계: 동시 접속자가 7,000 VUs를 초과할 경우 Redis Cluster 내부 네트워크 비용 증가로 인해 레이턴시가 3초 이상으로 지연됨.

추후에 시나리오 4 개선 방안 내용도 넣으면 좋을 것 같음
내용이 많아서 잘 정리해야 될 것 같음

1. 부하 테스트 및 성능 개선

부하 테스트 및 성능 개선

문생성, 결제요청
처리 과다

- 개선: 인덱싱, fetch 전략 조정, 비동기 이벤트 전환
- 결과: 응답시간/처리량/에러율 지표 개선(수치 삽입 권장)

로그 및 모니터링 전략

- 로그 레벨: INFO/WARN/ERROR 정책화
- 장애 대응: 에러 알람, SLA 임계치 기반 감지
- 추적: 요청 ID(TraceId) 기반 API-이벤트-DB 흐름 추적

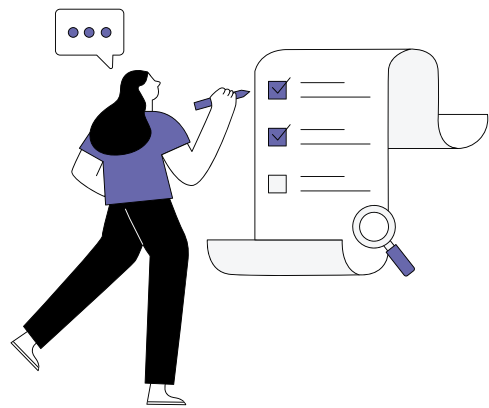
다이어그램 그림 필요

클라우드 인프라 설계

- AWS 기준: VPC, RDS, ElastiCache, MSK(or Kafka), EKS/k3s
- CI/CD: GitHub Actions → 이미지 빌드 → 레지스트리 → 배포
- IaC/자동화 기반으로 배포 리드타임 단축



Contents



01. 프로젝트 개요	02. 협업 및 개발 프로세스	03. MSA 아키텍처
04. DUKKU 시연 영상	05. 핵심 기능	06. 인프라 구축 및 시스템 고도화
07. 한계점 및 향후 개선 방향	08. 팀 구성과 역할	09. KPT 회고

Keep (잘한 점 / 유지할 것)

- DDD 기반 도메인 경계 설정으로 Modular Monolith → MSA 확장 가능 구조 확보
- Saga 기반 보상 트랜잭션으로 분산 정합성 문제 해결
- Idempotency-Key + Redis 상태 관리로 동시성 문제 안정화
- Refresh Rotation + Absolute Expiration으로 토큰 보안 강화
- Kafka 이벤트 기반 구조로 서비스 간 결합도 감소 및 장애 격리 달성

Problem (한계 및 아쉬운 점)

- Outbox 패턴 미적용으로 이벤트 신뢰성 100% 보장 어려움
- Kafka 재처리/리밸런싱 상황에 대한 시뮬레이션 테스트 부족
- 자동 제재(이상행동 탐지) 로직 미구현
- Coupon/정산 등 복잡 시나리오에 대한 통합 테스트 부족
- API Gateway 미도입으로 인증/관측성/로깅 통합 한계

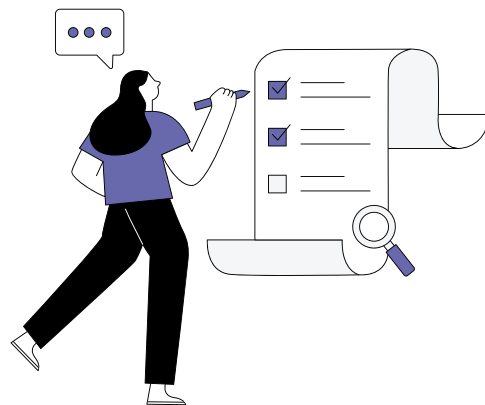
- 자동 제재 탐지 미구현 → 이상행동 탐지 도입 예정
- AI 고도화 필요 → 판매자 AI 어시스턴트 확장
- Outbox 패턴 미적용 → 메시지 신뢰성 강화 계획
- 쿠폰 시나리오 테스트 부족 → 케이스 4종 이상 보강
- API Gateway 미도입 → 인증/라우팅/관측성 통합 예정

- 쿠폰 시나리오 테스트 부족 → 케이스 4종 이상 보강
- API Gateway 미도입 → 인증/라우팅/관측성 통합 예정

Try (향후 개선 계획)

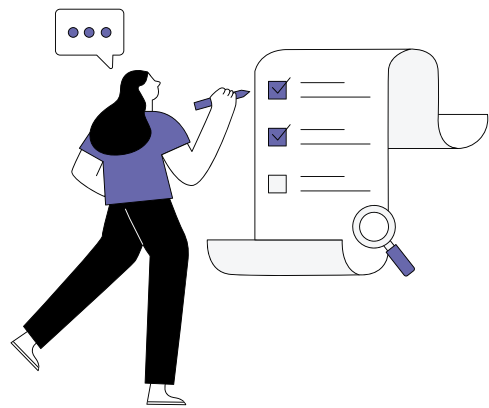
- Outbox 패턴 도입으로 DB-이벤트 발행 원자성 확보
- Kafka DLQ 및 재처리 전략 고도화
- 이상행동 탐지 기반 자동 제재 시스템 도입
- Coupon/정산 시나리오 통합 테스트 4종 이상 추가
- API Gateway 도입으로 인증·라우팅·관측성 통합
- OpenTelemetry 기반 분산 추적 도입 검토

Contents



01. 프로젝트 개요	02. 협업 및 개발 프로세스	03. MSA 아키텍처
04. DUKKU 시연 영상	05. 핵심 기능	06. 인프라 구축
07. 시스템 고도화	08. 팀 구성과 역할	09. KPT 회고

Contents



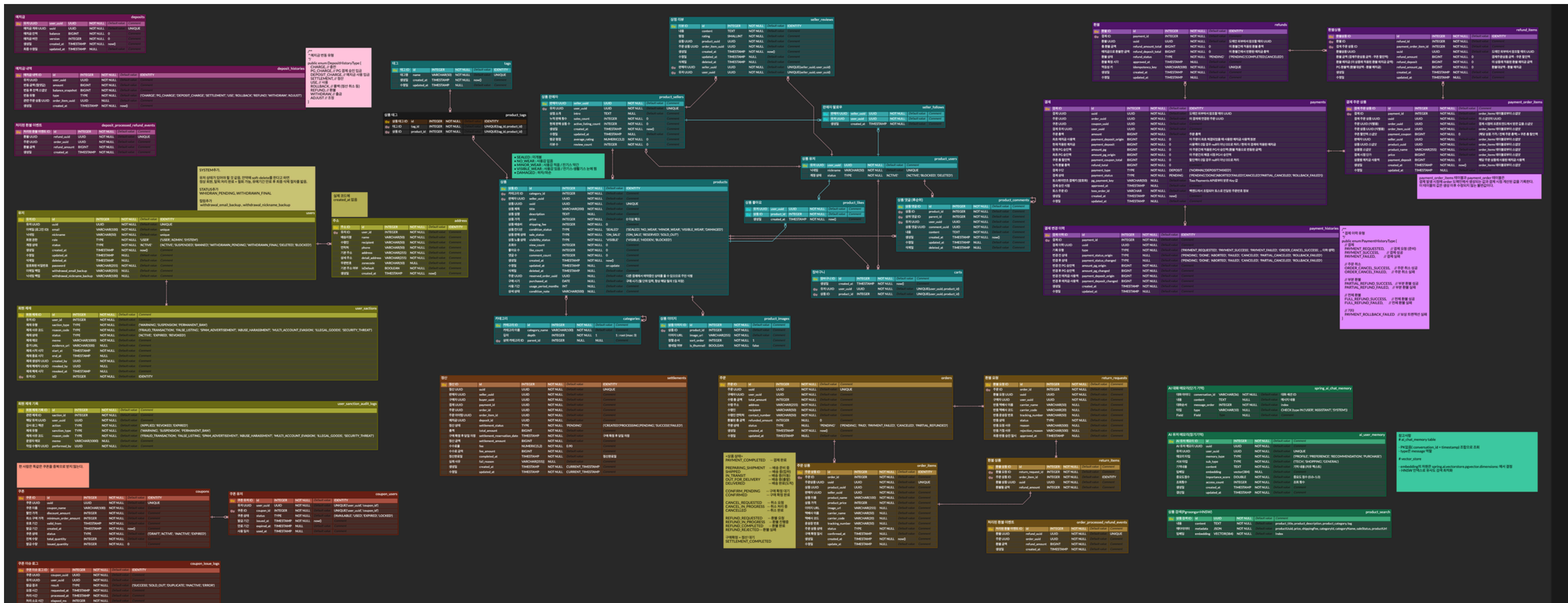
01. 프로젝트 개요	02. 협업 및 개발 프로세스	03. MSA 아키텍처
04. DUKKU 시연 영상	05. 핵심 기능	06. 인프라 구축 및 시스템 고도화
07. 한계점 및 향후 개선 방향	08. 팀 구성과 역할	09. KPT 회고

담당자

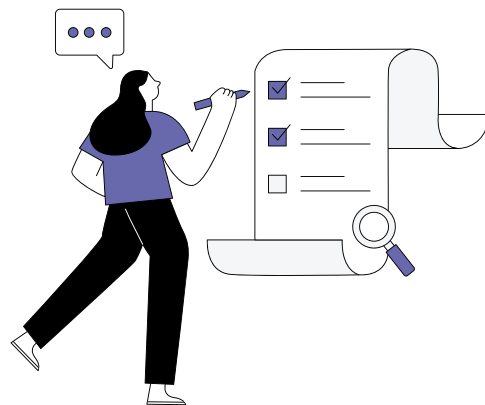
주요 담당 도메인 및 기술 스택

- 이태석 **Auth/User:** JWT 및 Redis Blacklist를 활용한 무상태(Stateless) 보안 체계 구축
- 박재현 **Payment/Deposit:** SAGA 패턴을 적용한 이벤트 기반 주문 - 결제 프로세스 구현 및 실패 시 안정적 복원을 위한 보상 트랜잭션 구조 구현
- 유승현 **AI/Settlement:** 사용자 취향 분석 데이터 처리 및 월 단위 정산 배치(Batch) 프로세스 안정성 확보
- 문찬희 **Order/Coupon:** 주문 시점의 상품 데이터 스냅샷 보존 및 동시성 이슈를 고려한 실시간 쿠폰 발급 로직
- 오세인 **Product/Infra:** Elasticsearch 기반 고성능 검색 인덱싱 및 Kubernetes를 통한 무중단 배포 환경 관리

팀 구성과 역할



Contents



01. 프로젝트 개요	02. 협업 및 개발 프로세스	03. MSA 아키텍처
04. DUKKU 시연 영상	05. 핵심 기능	06. 인프라 구축 및 시스템 고도화
07. 한계점 및 향후 개선 방향	08. 팀 구성과 역할	09. KPT 회고

운영 관점에서 설계한 안정성 포인트

이벤트 실패 시 문제

- 컨슈머 장애 → 이벤트 처리 지연/반복 실패
- 일부 도메인 상태 변경 → 상태 불일치
- 전체 이벤트 파이프라인 중단 위험

DLT(Dead Letter Topic) 격리

- 반복 실패 이벤트를 별도 토픽으로 분리
- 정상 이벤트 흐름과 완전히 분리 처리

서비스 전체 영향 최소화

- 실패 이벤트가 전체 소비를 막지 않음
- 관리자 개입 → 원인 분석 및 재처리 가능



실패를 숨기지 않고, 관리 가능한 구조로 격리



Keep – 현재 구조의 강점

- Spring 기반의 Modular Monolith 구조로 빠른 MVP 구현
- CQRS + Spring Event 기반 비동기 처리로 읽기/쓰기 분리
- Redis, Elasticsearch 도입으로 캐시/검색 성능 최적화
- 단일 배포 구조로 관리가 간편하며 진입장벽이 낮음

Problem – 한계 및 개선 필요 요소

- 도메인 간 논리적 분리는 존재하지만, 실제 배포 단위는 단일
→ 서비스 독립성 부족
- 트래픽 급증 시 스케일링 유연성 한계 (전체 redeploy 필요)
- Spring Event는 인메모리 기반 → 분산 시스템에서 신뢰성 낮음
- 운영/모니터링 측면에서 로그/이벤트 추적 미설계

Try – 향후 확장 계획 (MSA 전환)

A. 인프라 & 오케스트레이션 개선

- 각 도메인을 Docker 기반 개별 서비스로 분리
- Kubernetes(K8s) 도입
 - 오토스케일링, 장애 복구, 배포 자동화

B. DB 전략 변화 – 서비스별 독립 DB
(Database per Service)

- 각 마이크로서비스는 자신만의 DB만 접근
- 서비스 간 데이터 공유
 - API 호출 또는 이벤트 기반 메시징 처리
→ 결합도 최소화

C. 고급 통신 방식 도입

- 비동기 (Kafka):
 - Spring Event → Kafka 기반 Event Streaming으로 전환
 - 안정적인 분산 메시징 및 대용량 처리
- 동기 (FeignClient):
 - 마이크로서비스 간 HTTP 통신을 Spring Cloud OpenFeign으로 관리
 - 로드밸런싱, 장애 대응 (Circuit Breaker) 자동화

D. 인증 시스템 고도화
(AuthService)

- 로그아웃 처리
 - JWT Blacklist + Redis TTL 조합
- Stateless 구조 유지하면서 보안성 강화



5팀 세미콜론 : 이태석, 문찬희, 박재현, 유승현

발표자 : 오세인

덕꾸들을 위한
취향 기반 큐레이션 중고거래 플랫폼



서비스 링크 <https://dukku.shop/>

스웨거 링크 <https://api.dukku.shop/swagger-ui/index.html#/>

목업 [덕꾸(DUKKU) - 취미 중고거래]

목업/admin [Semicolon/admin - 취미 중고거래]